Context Management

Table of Contents

- 1. Overview
  - 1.1 Architecture Layers Overview
  - 1.2 System Design Principles
- 2. Token Counting System
  - 2.1 Precise Counting vs. Estimation
  - 2.2 Rough Estimation (When Precise Data Is Unavailable)
  - 2.3 Hybrid Counting (Standard Functions)
  - 2.4 Special Content Type Estimation
  - 2.5 Haiku-Assisted Counting
  - 2.6 Bedrock Special Handling
- 3. Context Window Management
  - 3.1 Core Constants
  - 3.2 Model Context Window Calculation
  - 3.3 Output Token Management
  - 3.4 Threshold Percentage System
  - 3.5 Blocking Limit
- 4. Microcompact
  - 4.1 Two Trigger Modes
  - 4.2 Microcompact Decision Flow
  - 4.3 Microcompact Warning Suppression
  - 4.4 API Layer Injection
- 5. Snip History Pruning
  - 5.1 How It Works
  - 5.2 Relationship with Compaction
  - 5.3 Snip Preservation Strategy
  - 5.4 Snip Trigger Conditions
- 6. API Layer Context Management
  - 6.1 Strategy Types
  - 6.2 Use Cases
- 7. Full Compaction
  - 7.1 Trigger Conditions
  - 7.2 Complete Execution Flow
  - 7.3 Compaction Summary Prompt Design
  - 7.4 Image Stripping
  - 7.5 Post-Compaction File Recovery
  - 7.6 Skill Recovery
  - 7.7 PTL Retry Mechanism
  - 7.8 Prefer Forked Sub-Agent Path During Compaction

    - 7.9 Direct Streaming Path (Fallback)
- 8. Session Memory
    - 8.1 Core Difference from Compaction
    - 8.2 Trigger Conditions
    - 8.3 Sub-Agent Execution Flow
    - 8.4 Session Memory Template Explained
    - 8.5 Token Limits
    - 8.6 Custom Template Support
    - 8.7 Tool Usage Restrictions
    - 8.8 Mutual Exclusion with Main Agent
    - 8.9 Usage of Session Memory
- 9. Session Memory Compaction (Important)
    - 9.1 Comparison with Traditional Compaction
    - 9.2 Preserved Message Calculation
    - 9.3 Feature Usage Determination
    - 9.4 Execution Flow
    - 9.5 Compaction Boundary Metadata
- 10. Auto-Compact
    - 10.1 Threshold System Explained
    - 10.2 Warning States
    - 10.3 Compaction Priority
    - 10.4 Circuit Breaker Explained
    - 10.5 Disable Control Matrix
    - 10.6 Special Nature of Marble Origami
- 11. Context Collapse
    - 11.1 Core Difference from Compaction
    - 11.2 Design Principles
    - 11.3 Mutual Exclusion with Auto-Compact
    - 11.4 Implementation Location
    - 11.5 Objects Being Collapsed
    - 11.6 Two-Phase Collapse Algorithm
    - 11.7 Relationship with Other Mechanisms
- 12. Context Analysis
    - 12.1 Use Cases
    - 12.2 Statistical Dimensions
    - 12.3 Duplicate Read Detection Algorithm
    - 12.4 Statsig Metric Mapping
    - 12.5 Detectable Waste Points
    - 12.6 Summary
- 13. Post-Compaction Cleanup
    - 13.1 Cleanup Content Explained
    - 13.2 Key Items NOT Cleared

    - 13.3 Main Thread vs. Sub-Agent Distinction
- 14. Context Loading and Caching
    - 14.1 System Context
    - 14.2 Session Storage
- 15. Persistent Memory Extraction
    - 15.1 Difference from Session Memory
    - 15.2 Flow
    - 15.3 Storage Structure and Content
    - 15.4 Sub-Agent Execution
    - 15.5 Mutual Exclusion Protection
    - 15.6 Wait on Exit
- 16. History Management
    - 16.1 Key Constants
    - 16.2 Main Functions
- 17. Message Normalization and Boundary Management
    - 17.1 Message Normalization
    - 17.2 Compaction Boundary Messages
    - 17.3 Microcompact Boundary Messages
    - 17.4 Finding Compaction Boundaries
- 18. Key Design Decisions
    - 18.1 Why Fork a Sub-Agent for Summary?
    - 18.2 Why Microcompact at the API Layer Instead of Directly Modifying Messages?
    - 18.3 Why Use GrowthBook Dynamic Configuration?
- 19. Global Data Flow Overview
    - 19.1 Complete Request Processing Flow
    - 19.2 Compaction Message Construction Flow
    - 19.3 Token Consumption by Context Layer

---

1. Overview

Claude Code's context management system is one of the most core and complex subsystems in the entire project. It is composed of multiple layers of mechanisms, ranging from low-level token counting, to mid-level microcompaction (tool result pruning), to high-level full compaction (conversation history summarization), and finally session-level and persistent-level memory management. The entire system revolves around one core goal: **maximizing effective information within the limited context window**.

1.1 Architecture Layers Overview

```
┌──────────────────────────────────────────────────────────┐
│  Layer 5: Persistent Memory Extraction (extractMemories) │
│            → Cross-session persistence to ~/.claude/projects/ │
├──────────────────────────────────────────────────────────┤
│  Layer 4: Session Memory                                 │
│            → Current session Markdown notes              │
├──────────────────────────────────────────────────────────┤
```

```
│  Layer 3: Full Compaction                          │
│            → AI summarized conversation history (model call) │
├──────────────────────────────────────────────────────┤
│  Layer 2: History Pruning (Snip)                   │
│            → Selectively discard history (by time/token) │
├──────────────────────────────────────────────────────┤
│  Layer 1: Microcompact                             │
│            → Clean old tool results, keep prompt cache │
├──────────────────────────────────────────────────────┤
│  Layer 0: Token Counting and Threshold Detection    │
│            → Mathematical foundation for all decisions │
└──────────────────────────────────────────────────────┘
```

1.2 System Design Principles

- **Layered Management**: Each layer handles problems of different granularity (Token-level → Tool-level → Message-level → Session-level)
- **Conservative Triggering**: All automatic mechanisms have circuit breakers to prevent wasted API calls from repeated failures
- **Zero Information Loss Intent**: Compress history through structured summarization rather than simple discarding
- **Prompt Cache Friendly**: Compaction operations preferentially reuse existing API prompt caches
- **Progressive Optimization**: Prefer lightweight mechanisms first (MC → Snip (disabled by default) → Compaction), with heavyweight mechanisms as fallback

---

2. Token Counting System

Token counting is the foundation of the entire context management system, and all compaction decisions are based on token counts.

2.1 Precise Counting vs. Estimation

Claude Code uses two token counting strategies, each with its own applicable scenarios:

Precise Counting (After API Call)

```ts
// utils/tokens.ts
getTokenUsage(message: Message): Usage | undefined

// Extracts from the usage field of assistant messages
// Conditions: message has usage field + not a synthetic message + not SYNTHETIC_MODEL
// Does not return progress/attachment/system type messages
```

The `Usage` object contains:

```ts
{
  input_tokens: number,
  output_tokens: number,
  cache_creation_input_tokens?: number,  // Cache creation cost
  cache_read_input_tokens?: number,       // Cache read savings
}
```

Precise Total Context Window Token Count

```
// utils/tokens.ts
getTokenCountFromUsage(usage: Usage): number
// = input_tokens + cache_creation_input_tokens + cache_read_input_tokens + output_tokens
// This number represents the full context size at the time of this API call
```

Getting Precise Count from Historical Messages

```
// utils/tokens.ts
tokenCountFromLastAPIResponse(messages: Message[]): number
// Traverses forward through the message array, finds the first assistant message with usage
// Returns that message's getTokenCountFromUsage
// Used for comparing token counts before and after compaction
```

Final Window Tokens for task_budget

```
// utils/tokens.ts
finalContextTokensFromLastResponse(messages: Message[]): number
// Gets final window size from usage.iterations[-1]
// (Used in server-side tool loop scenarios)
// Formula: input_tokens + output_tokens (excluding cache tokens)
```

2.2 Rough Estimation (When Precise Data Is Unavailable)

```
// services/tokenEstimation.ts
roughTokenCountEstimation(content: string, bytesPerToken = 4): number
// Simplest estimation: character count ÷ 4
// Suitable for plain text content
```

Ratio Adjustment by File Type:

```
// JSON/JSONL/JSONC files have a byte/token ratio close to 2 (due to many single-character
symbols)
// Other files use the default 4
bytesPerTokenForFileType(fileExtension: string): number {
  switch (fileExtension) {
    case 'json': case 'jsonl': case 'jsonc': return 2
    default: return 4
  }
}
```

2.3 Hybrid Counting (Standard Functions)

```
// utils/tokens.ts
tokenCountWithEstimation(messages: readonly Message[]): number
```

This is the **most important counting function**, serving as the basis for auto-compaction threshold decisions. The logic is as follows:

PS: Use precise calculation where possible; for parts that cannot be precisely calculated, use rough estimation separately.

```
1. Traverse backward from the last message
2. Find the first assistant message with precise usage data
3. If found:
   a. First, traverse forward and include all split messages with the same message.id in the estimate
      (When parallel tool calls produce multiple assistant messages from one API response)
   b. Precise usage + rough estimation for subsequent messages
4. If not found: use rough estimation for everything
```

Handling Parallel Tool Calls: When the model makes multiple tool calls in parallel within one response, the streaming code emits separate assistant message records for each content block (sharing `message.id` and `usage`), and immediately inserts tool_results between each tool_use. The message array structure is as follows:

```
[..., assistant(id=A, usage), user(tool_result_A), assistant(id=A, usage),
 user(tool_result_B), assistant(id=A, usage), user(tool_result_C), ...]
```

If we stop only at the last assistant record, we miss all the intermediate inserted tool_results, leading to underestimation. The backward-traversal logic in `tokenCountWithEstimation` includes all records sharing the same `message.id` in the estimate.

PS:

- **Blind spot in conventional calculation logic**: Many statistical systems, when calculating token consumption for a conversation turn or assembling historical messages, habitually **"look for the last assistant message"** as the endpoint of the current turn.
- **Why underestimation occurs**: If the system only grabs the last `assistant(id=A, usage)` record for settlement, it will ignore all the previous `assistant` records that also belong to `id=A`, and will miss the huge text in between — that is, the `tool_result` (tool return results usually contain many tokens). This causes the system to calculate far fewer tokens than actually consumed (underestimation).

2.4 Special Content Type Estimation

```ts
// services/tokenEstimation.ts
roughTokenCountEstimationForBlock(block): number

// Text block: character count ÷ 4
if (block.type === 'text')
  return roughTokenCountEstimation(block.text)

// Image/document block: fixed 2000 tokens
// Reason: Claude platform's billing formula for images is (width × height) / 750
// Maximum 2000×2000 = 5333 tokens, conservative value of 2000
if (block.type === 'image' || block.type === 'document')
  return 2000  // IMAGE_MAX_TOKEN_SIZE

// tool_use: tool name + JSON-serialized input parameters
// Note: input here could be arbitrarily large (bash commands, file contents, etc.)
// Directly stringify once for estimation, because the API will re-serialize anyway
```

```
if (block.type === 'tool_use')
  return roughTokenCountEstimation(
    block.name + JSON.stringify(block.input ?? {})
  )

// tool_result: recursively calculate its content
if (block.type === 'tool_result')
  return roughTokenCountEstimationForContent(block.content)

// thinking block: only count thinking text, not JSON wrapper and signature
if (block.type === 'thinking')
  return roughTokenCountEstimation(block.thinking)

// Redacted thinking: count the data field
if (block.type === 'redacted_thinking')
  return roughTokenCountEstimation(block.data)

// All other block types (server_tool_use, web_search_tool_result, etc.)
// Unified stringify then estimate
return roughTokenCountEstimation(JSON.stringify(block))
```

2.5 Haiku-Assisted Counting

```
// services/tokenEstimation.ts
countTokensViaHaikuFallback(messages, tools): Promise<number>
```

When precise usage is unavailable (e.g., after cross-session recovery), Claude Code uses Haiku (the fastest model) to estimate token counts:

```
const model = isVertexGlobalEndpoint || isBedrockWithThinking || isVertexWithThinking
  ? getDefaultSonnetModel()   // Vertex global endpoint or Bedrock+thinking: use Sonnet
  : getSmallFastModel()         // Otherwise use Haiku 4.5 (supports thinking)
// Calls anthropic.beta.messages.create (max_tokens=1 or 2048)
// Returns input_tokens + cache_creation + cache_read
```

2.6 Bedrock Special Handling

```
// services/tokenEstimation.ts
countTokensWithBedrock({ model, messages, tools, betas, containsThinking })

// Bedrock's CountTokens API requires the base model ID (not the inference profile ARN)
// Needs conversion via getInferenceProfileBackingModel()
// API model: bedrock-2023-05-31
```

---

3. Context Window Management

3.1 Core Constants (`utils/context.ts`)

```
MODEL_CONTEXT_WINDOW_DEFAULT = 200_000    // Default context window: 200K tokens
COMPACT_MAX_OUTPUT_TOKENS = 20_000       // Max output for compaction model calls
```

```
ESCALATED_MAX_TOKENS = 64_000            // Escalated mode max output
CAPPED_DEFAULT_MAX_TOKENS = 8_000         // Default capped mode max output
```

3.2 Model Context Window Calculation

```
getContextWindowForModel(model: string, betas?: string[]): number
// Standard models: 200K
// Models supporting 1M (via has1mContext / modelSupports1M): 1,000,000
// Special beta configurations: may be smaller

getEffectiveContextWindowSize(model: string): number
// = getContextWindowForModel(model) - MAX_OUTPUT_TOKENS_FOR_SUMMARY(20K)
// Reserves 20K for model output, actual available input window
// Supports environment variable override: CLAUDE_CODE_AUTO_COMPACT_WINDOW=150000
```

3.3 Output Token Management

```
getModelMaxOutputTokens(model: string): number
// Different models have different output caps:
// Sonnet 4.6: 64K
// Sonnet 4.5: 32K
// Haiku: 8K
// ...
```

3.4 Threshold Percentage System

```
Effective Context Window (effectiveWindow)
  ┌──────────────────────────────────────────┬────────────────┐
  │                                          │                │ ← Output Buffer (20K)
  │  0%              Warning Line    93.5%│  100%   100% │
  │     ├────────────────┤├────────────────┤   │              │
  │     WARNING_BUFFER  AUTO_BUFFER        │  Overflow!    │
  │      (20K)         (13K)              │              │
  └──────────────────────────────────────────┴────────────────┘

          ↑
    calculateTokenWarningState(tokenUsage, model)
    Returns:
      percentLeft        = remaining percentage
      isAboveWarningThreshold      // Warning
      isAboveErrorThreshold        // Error
      isAboveAutoCompactThreshold  // Auto-compact
      isAtBlockingLimit            // Block
```

3.5 Blocking Limit

```
// Beyond this threshold, block user input (prevent overflow)
isAtBlockingLimit = tokenUsage >= blockingLimit
// blockingLimit = effectiveWindow - MANUAL_COMPACT_BUFFER_TOKENS(3K)
// approximately effectiveWindow - 3K

// Supports environment variable override: CLAUDE_CODE_BLOCKING_LIMIT_OVERRIDE
```

4. Microcompact

Microcompact is the lightest-weight context management mechanism executed before each API call. It does NOT perform summarization, but instead directly cleans or replaces the content of old tool results.
PS: In essence, it prunes tool results.

4.1 Two Trigger Modes

4.1.1 Time-Based Microcompact

Core Idea: Claude API's Prompt Cache has a 1-hour (60-minute) TTL. When the time since the last Assistant message exceeds 60 minutes, **the cache has expired**, and the entire prefix will be recalculated. At this point, **cleaning old tool results in advance can reduce the amount of data to be rewritten**.

PS: Whether the cache hits depends not only on whether the prefix is the same, but also on the TTL situation. However, this **TTL is timed by measuring the time difference between two requests** (when a new request arrives, the timer resets).

```
// services/compact/timeBasedMCConfig.ts
const TIME_BASED_MC_CONFIG_DEFAULTS = {
  enabled: false,               // Disabled by default, enabled via GrowthBook
'tengu_slate_heron'
  gapThresholdMinutes: 60,      // 60 minutes (aligned with server cache TTL)
  keepRecent: 5,                // Keep the 5 most recent compactable tool results
}
```

Trigger Evaluation:

```
evaluateTimeBasedTrigger(messages, querySource): { gapMinutes, config } | null
// Must satisfy:
// 1. config.enabled = true (GrowthBook switch)
// 2. querySource === 'repl_main_thread' (main thread only)
// 3. Has a recent assistant message
// 4. gap = (now - lastAssistant.timestamp) / 60000 >= config.gapThresholdMinutes
```

Processing Logic (`maybeTimeBasedMicrocompact`):

1. `collectCompactableToolIds()` collects tool_use_ids of all **compactable tools** (in order of appearance) -> Also sets which tool results are compactable. Key tools like AgentTool and TaskOutputTool are **NOT in this list** by default and won't be cleaned
2. Keep the most recent N (`keepRecent`), add the rest to `clearSet` -> Pruning protection
3. Iterate through messages, replacing content of tool_results belonging to tools in `clearSet` with `[Old tool result content cleared]`
4. **Key**: Modifying prompt content breaks the server-side cache, so reset Cached MC state and notify cache break detection -> Cache has already expired, so there won't be cost increases from KV cache misses

PS: Batch pruning in one operation is more reasonable than OpenClaw's dynamic balancing via window guards.

Cleanup Scope (`COMPACTABLE_TOOLS`):

```
const COMPACTABLE_TOOLS = new Set([
  'Read',
```

```
  'Bash', 'PowerShell',  // ...all SHELL_TOOL_NAMES
  'Grep',
  'Glob',
  'WebSearch',
  'WebFetch',
  'Edit',
  'Write',
])
```

4.1.2 Cached Microcompact

Core Idea: Utilizing the API's `cache_edits` mechanism, by specifying `cache_reference` (referencing existing cached segments) and `cache_edits` (edit operations) in the request, **old tool results are deleted** while **preserving the validity of the prompt cache**.

PS:

- Advantage: Using cache_edit for deletion effectively masks part of the KV, improving context quality without breaking the KV cache
- These are fields considered in session modeling; they are not in context (all infra-level directive parameters are in the session, not in context)

```
┌──────────────────────────────────────────────────────────────┐
│  Server-side Cache (Prompt Cache)                            │
│                                                              │
│  [System prompt + Tool definitions + Message prefix]   ← Cached, cache_hit │
│                                                              │
│  [Tool result A: Read foo.py]           ← cache_edit: delete │
│  [Tool result B: Bash git status]      ← cache_edit: delete │
│  [Tool result C: Grep "func"]           ← kept               │
│  [Latest user message]                                       │
└──────────────────────────────────────────────────────────────┘
                              ↓
                     API request only transmits delta
                     (what to delete, not everything)
```

Enable Conditions (`microCompact.ts` 's `cachedMicrocompactPath`):

1. Feature flag `CACHED_MICROCOMPACT` is enabled
2. Model supports cache editing (`isModelSupportedForCacheEditing(model)`) -> Checks if the model service supports this capability (decided by infra side)
3. `isMainThreadSource(querySource)` is true (`querySource === 'repl_main_thread'` or starts with this prefix)

State Tracking (`cachedMicrocompact.js` , dynamic import):

```
CachedMCState = {
  toolOrder: string[],              // All registered tool_use_ids (in order)
  registeredTools: Set<string>,    // Registered tool_use_ids
  deletedRefs: Set<string>,         // Deleted tool_use_ids
  pinnedEdits: PinnedCacheEdits[],  // Pinned edit operations
}
```

4.2 Microcompact Decision Flow

```
// services/compact/microCompact.ts
microcompactMessages(messages, toolUseContext?, querySource?): MicrocompactResult

// Execution order (short-circuit logic):
async function microcompactMessages(messages, toolUseContext, querySource) {
  clearCompactWarningSuppression()  // Clear warning suppression

  // 1. Time-based MC takes priority (detects cache has expired)
  const timeResult = maybeTimeBasedMicrocompact(messages, querySource)
  if (timeResult) return timeResult  // Short-circuit: time trigger takes priority

  // 2. Cache edit MC (must meet conditions)
  if (feature('CACHED_MICROCOMPACT') && isCachedMCEnabled() && isModelSupported() &&
isMainThread) {
    return cachedMicrocompactPath(messages, querySource)  // Short-circuit
  }

  // 3. No available MC path → return original messages (fallback)
  return { messages }
}
```

4.3 Microcompact Warning Suppression

```
// services/compact/compactWarningState.ts
compactWarningStore: Store<boolean>

// Set suppression=true immediately after successful microcompact
// UI layer subscribes to this state via useCompactWarningSuppression() hook
// When suppression=true, the "context about to compact" warning is not shown
// Reason: After microcompact, the client doesn't know how large the context actually is;
// the next API response has precise data, so without knowing the specific value,
// the "context about to compact" warning is not shown
```

4.4 API Layer Injection

During API request construction (`microCompact.ts` → caller), pending `cache_edits` are retrieved via `consumePendingCacheEdits()` and injected into the request. At the same time, `cache_deleted_input_tokens` is extracted from the last assistant message as the baseline for calculating the delta.

---

5. Snip History Pruning (Similar to context truncation, disabled by default)

Snip (`HISTORY_SNIP` feature flag) is a mechanism that directly discards early content from the message history without summarization. Unlike compaction, Snip discards content the model no longer needs to see (e.g., file read results that have already been processed).

5.1 How It Works

```
Message History
  [msg0] [msg1] ... [msgN-2] [msgN-1] [latest msg]
      ↑                               ↑
   snip point ← ← ← ← ← ← ← ← ← ← ←  keep tail

After Snip:
  [SNIP_MARKER] [msgN-2] [msgN-1] [latest msg]
```

5.2 Relationship with Compaction

- **Can coexist**: Snip and compaction are not mutually exclusive and can trigger simultaneously
- **Execution order**: Snip executes **before** microcompact (`query.ts:396`)
- **Token release tracking**: `snipTokensFreed` is passed to `autoCompactIfNeeded` to ensure auto-compaction threshold decisions reflect the space already freed by Snip

```
// In query.ts:
if (feature('HISTORY_SNIP')) {
  const snipResult = snipModule!.snipCompactIfNeeded(messagesForQuery)
  messagesForQuery = snipResult.messages
  snipTokensFreed = snipResult.tokensFreed
  if (snipResult.boundaryMessage) {
    yield snipResult.boundaryMessage
  }
}
// ... subsequent microcompact and autocompact use snipTokensFreed
```

5.3 Snip Preservation Strategy

Snip does not discard indiscriminately; it preserves:

1. **Most recent N messages**: Ensures working context is not broken
2. **Messages containing tool_use not yet paired with tool_result**: Prevents orphan tool_results
3. **Session boundary messages**: Such as compaction boundaries, SessionStart messages, etc.

PS:
Directly modifies messages[], replacing an intermediate segment with a placeholder marker, then only sends the later portion to the API -> Equivalent to truncating the head.

`` `is a dynamically loaded compiled module via require() for lazy loading. The specific trigger timing is unknown, but from the code structure it appears to be quantity-based (alternatively, users can manually trigger via the /force-snip` command).

5.4 Snip Trigger Conditions

1. Feature Flag switch

```
  // Conditional compilation switch — HISTORY_SNIP enabled only when true
  feature('HISTORY_SNIP')
```

2. Trigger Method 1: Automatic check per Query loop (automatic trigger)

```
User Input
    ↓
query.ts:401 —必经之路before each API call
    ↓
snipModule!.snipCompactIfNeeded(messagesForQuery)
    ↓
[Condition check] Is snip needed?
    ↓
Possible output: messages + tokensFreed + boundaryMessage

Core call site query.ts:401-410:
```typescript
if (feature('HISTORY_SNIP')) {
  const snipResult = snipModule!.snipCompactIfNeeded(messagesForQuery)
  messagesForQuery = snipResult.messages   // Pruned messages
  snipTokensFreed = snipResult.tokensFreed  // Tokens freed
  if (snipResult.boundaryMessage) {
    yield snipResult.boundaryMessage        // Yield boundary marker
  }
}
```
```

Execution order: Executes before microcompact (query.ts:396 comment)

3. Trigger Method 2: Model proactively calls snip tool (tool call trigger)

SNIP is exposed to the model as a tool (SnipTool):

```typescript
// tools.ts:123
const SnipTool = feature('HISTORY_SNIP')
  ? require('./tools/SnipTool/SnipTool.js').SnipTool
  : null
```

The model can proactively call the snip tool in any turn, telling Claude Code which history segment to prune. Tool call triggers `snipCompactIfNeeded({ force: true })` (see QueryEngine.ts:1238 snipReplay logic).

4. Trigger Method 3: Manual command /snip

```typescript
// commands.ts:83
const forceSnip = feature('HISTORY_SNIP')
  ? require('./commands/force-snip.js').default
  : null
```

When HISTORY_SNIP is enabled, users can manually call the /snip command to force pruning.

5. Nudge Mechanism: Periodic reminders for the model to use snip

```typescript
// attachments.ts:3963
export function getContextEfficiencyAttachment(messages: Message[]): Attachment[] {
  if (!isSnipRuntimeEnabled()) return []
  if (!shouldNudgeForSnips(messages)) return []  // ← Trigger check
  return [{ type: 'context_efficiency' }]         // ← Inject reminder
}
```

---

6. API Layer Context Management

When the model API **supports native context management strategies**, you can **declare context management strategies in the context_management field of the request**, and the server's inference engine automatically performs cleanup operations.

PS: The essence is **partially delegating context management decisions to the server**, allowing the server to automatically handle them at appropriate times, without the client repeatedly sending cache_edits or doing content replacement.

6.1 Strategy Types (`services/compact/apiMicrocompact.ts`)

```
type ContextManagementConfig = {
  edits: ContextEditStrategy[]
}
```

6.1.1 `clear_tool_uses_20250919`

```
{
  type: 'clear_tool_uses_20250919',
  trigger: {
    type: 'input_tokens',
    value: 180_000  // Trigger threshold (DEFAULT_MAX_INPUT_TOKENS)
  },
  keep: {
    type: 'tool_uses',
    value: N         // Keep the most recent N tool calls
  },
  clear_tool_inputs: [
    'Bash', 'Read', 'Grep', 'Glob', 'WebSearch', 'WebFetch'
  ],
  clear_at_least: {
    type: 'input_tokens',
    value: 140_000  // Clear at least 140K tokens (180K - 40K)
  },
  exclude_tools: ['Edit', 'Write', 'NotebookEdit']  // Don't clear edit-type tools
}
```

Trigger timing: When `input_tokens >= trigger.value`, the server automatically executes cleanup up to the `keep` limit.

6.1.2 `clear_thinking_20251015`

```
{
  type: 'clear_thinking_20251015',
  keep: 'all' | { type: 'thinking_turns', value: N }
// Keep the most recent N thinking blocks (or all)
// Skip when redact-thinking is active (because redacted blocks have no model-visible
content)
}
```

6.2 Use Cases

- **Thinking cleanup**: Used by default in all sessions with thinking that don't have **redact-thinking** enabled
- **Tool result cleanup**: Enabled via `USE_API_CLEAR_TOOL_RESULTS` / `USE_API_CLEAR_TOOL_USES` environment variables (ANT users only)

PS:

- redact-thinking: "Redacts" thinking blocks — replaces model-generated thinking content with unreadable redacted_thinking blocks. Mainly for privacy/security and token billing concerns
- ANT users: Abbreviation for Anthropic internal employees, can test features still in experimentation

---

7. Full Compaction

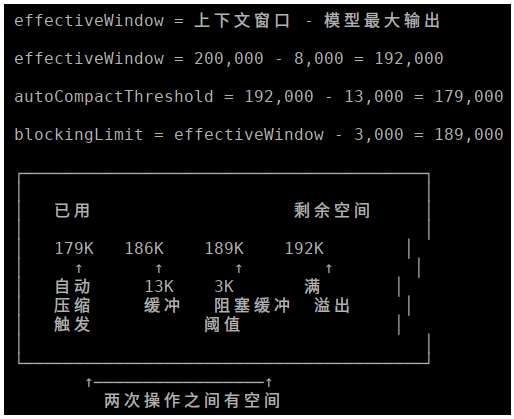
When all lightweight mechanisms are insufficient to control context size, Claude Code performs full compaction: generating a structured summary using an AI model and replacing the entire conversation history.

7.1 Trigger Conditions

Triggered by token count:

```
tokenCountWithEstimation(messages) >= getAutoCompactThreshold(model)
// = effectiveContextWindow - AUTOCOMPACT_BUFFER_TOKENS(13K)
// Approximately 93.5% of effectiveWindow
```

PS: effectiveWindow = model context window - model max output tokens



7.2 Complete Execution Flow

```
Input: messages[] (current full conversation history)
Output: CompactionResult

Phase 1: Pre-processing
  ↓
  executePreCompactHooks()  ← Execute pre-compaction hooks (allows custom directives)
  ↓
Phase 2: Generate Summary (call AI model)
  ↓
  stripImagesFromMessages()   ← Strip images/documents
  stripReinjectedAttachments()← Strip reinjectable attachments (two types:
                                 skill_discovery signal (tells model available skills)
                                 and skill_listing content. Stripped because these
                                 attachments will be reinjected after compaction)
```

```
  ↓
  streamCompactSummary()       ← Call model to generate summary
      ├── Priority: fork sub-agent path (shared prompt cache)
      └── Fallback: direct streaming call (streaming fallback)
  ↓
  PTL retry (up to 3 times):
      truncateHeadForPTLRetry()  ← If the compaction request itself exceeds context
  ↓
Phase 3: State Cleanup
  ↓
  context.readFileState.clear()         ← Clear file read cache (after each Read tool call,
                                          file content is cached. Clearing is for ALL files
                                          not selective; reasons:
                                          1. Prevent duplication between compaction result
                                             and re-read content;
                                          2. Prevent file content modified by user or
                                             environment, force reading latest version;
                                          3. After compaction, conversation history becomes
                                             summary, model no longer needs "files the model
                                             claims to have read")

  context.loadedNestedMemoryPaths.clear()← Clear nested memory paths (this Set records which
                                          nested CLAUDE.md files have been injected into the
                                          conversation during this session; clearing is for
                                          deduplication)
  ↓
Phase 4: Reinject Attachments (post-compaction info backfill)
  ↓
  Keep recent state, maintain semantic continuity
  createPostCompactFileAttachments()    ← Restore recently accessed files (up to 5, total
50K)

  Restore state
  createAsyncAgentAttachmentsIfNeeded()  ← Restore async agent state
  createPlanAttachmentIfNeeded()         ← Restore Plan file
  createPlanModeAttachmentIfNeeded()    ← Restore Plan Mode reminder
  createSkillAttachmentIfNeeded()       ← Restore invoked skills (5K per skill), has skill
                                          invoked state

  Restore environment info
  getDeferredToolsDeltaAttachment()      ← Re-announce tool changes
  getAgentListingDeltaAttachment()        ← Re-announce agent list
  getMcpInstructionsDeltaAttachment()  ← Re-announce MCP instructions
  ↓
Phase 5: Create Compaction Boundary
  ↓
  createCompactBoundaryMessage()        ← Create system boundary message (update location
                                          marker, mark which content before what point
                                          was compacted)
  ↓
Phase 6: SessionStart Hooks
  ↓
  processSessionStartHooks('compact')  ← Reload CLAUDE.md and other context (doing this
```

```
                                          separately after creating compaction boundary
                                          because of different process modeling:
                                          tool/agent/MCP delta are built in real-time during
                                          compaction and placed in messages array;
                                          Memory/CLAUDE.md are frozen in systemPrompt
                                          before compactConversation() is called.
                                          If modified during compaction, it only takes
effect
                                          on next API request — so干脆等下一轮 turn.
                                          It's not for the current API call, but for the
next
                                          getSystemPrompt() call — ensures the next turn
gets
                                          fresh memory content)
  ↓
Phase 7: Event Logging and Cleanup
  ↓
  notifyCompaction()                    ← Notify cache break detection (to prevent cache
                                          break detection from false positives after
                                          compaction: after compaction, message count
                                          legitimately drops significantly,
cache_read_tokens
                                          will drop significantly — this is normal behavior,
                                          not a break)
  markPostCompaction()                  ← Set post-compaction state
  reAppendSessionMetadata()             ← Refresh session metadata
  writeSessionTranscriptSegment()       ← Write session transcript (KAIROS feature)
  executePostCompactHooks()             ← Execute post-compaction hooks
  runPostCompactCleanup()               ← Cleanup caches and state
```

7.3 Compaction Summary Prompt Design

The compaction summary requires the model to generate highly structured output, which is one of the most critical parts of the entire compaction system.

7.3.1 Prompt Template Structure

The prompt contains three parts:

1. Mandatory No-Tools Lead (CRITICAL)

```
CRITICAL: Respond with TEXT ONLY. Do NOT call any tools.
- Do NOT use Read, Bash, Grep, Glob, Edit, Write, or ANY other tool.
- You already have all the context you need in the conversation above.
- Tool calls will be REJECTED and will waste your only turn — you will fail the task.
- Your entire response must be plain text: an <analysis> block followed by a <summary>
block.
```

```
**CRITICAL**: Only reply with plain text. Do not call any tools.
- Do not use Read, Bash, Grep, Glob, Edit, Write, or any other tools.
- You already have all the context you need in the conversation above.
- Tool calls will be rejected and will waste your only turn — this will cause your task to
fail.
- Your entire response must be plain text: one block, followed by another block.
```

Design Rationale: The compaction fork sub-agent inherits the parent session's complete toolset, but the model might still attempt tool calls. This lead explicitly prohibits tool usage, ensuring the model only outputs text.

PS: During OpenClaw compaction prompt testing, it was found that the model has some probability of not following compaction instructions and instead choosing to continue executing subsequent tasks. Only when the compaction agent system prompt and query structure were changed did compaction instruction following improve. CC not having this issue may be due to model intelligence.

2. Detailed Analysis Instructions (DETAILED_ANALYSIS_INSTRUCTION)

The model is required to do the following in the `<analysis>` block:

- Analyze each message in chronological order -> Overcomes the model's "attention loss" and "context forgetting" problems in long conversations, preventing logical discontinuities
- Identify: explicit requests, intent, code snippets, filenames, function signatures -> Entity extraction, convert to structured information
- Record: encountered errors, user corrections, methods no longer attempted -> Negative memory, prevents repeated pitfalls
- Ensure technical accuracy and completeness

3. Summary Format Requirements (9 Required Parts)

PS: OpenClaw, being a more general agent, has similar other sections (mainly user request info, environment info, task status info), but **important code snippets** etc. are replaced with **important context snippets**

```
1. Primary Request and Intent:
   All explicit user requests and intent

2. Key Technical Concepts:
   List of important technical concepts, frameworks, patterns

3. Files and Code Sections:
   - File name
     - Why this file is important
     - What modifications were made to this file (if any)
     - Important code snippets
   - ...

4. Errors and Fixes:
   - Error description:
     - Fix method
     - User feedback on this error (if any)
   - ...

5. Problem Solving:
   Solved problems + ongoing investigation

6. All User Messages:
   All non-tool-result user messages (in chronological order)

7. Pending Tasks:
   Explicitly requested but not yet completed tasks
```

```
8. Current Work:
   Specific work in progress before compaction
   (includes filenames, code snippets)

9. Optional Next Step:
   Next step directly continuing the most recent work
   (must reference specific content from the original conversation)
```

7.3.2 Output Parsing

```
// Extract summary from model response in compact.ts:
const summary = getAssistantMessageText(summaryResponse)

// <analysis> block is stripped during parsing, only <summary> part is kept
// Because <analysis> is a draft process and should not enter the context
```

7.3.3 Summary Result Storage

```
// Summary is stored as a user message (role='user')
createUserMessage({
  content: getCompactUserSummaryMessage(summary, suppressQuestions, transcriptPath),
  isCompactSummary: true,              // Marked as compaction summary
  isVisibleInTranscriptOnly: true,     // Only visible in transcript, not in API
  summarizeMetadata: {
    messagesSummarized: N,
    userContext: string,
    direction: 'from' | 'up_to',
  }
})
```

Note: `isVisibleInTranscriptOnly` ensures the summary **does not enter the next API request** (filtered by normalizeMessagesForAPI), but does appear in the user-visible transcript.

7.4 Image Stripping

```
stripImagesFromMessages(messages: Message[]): Message[]
```

Images in user messages are replaced before being fed into compaction: (**placeholder replacement**)

- `image` blocks → `[image]` text blocks
- `document` blocks → `[document]` text blocks
- **Images/documents nested in tool results**: handled the same way

Rationale: Images don't directly help generate conversation summaries. But images would cause the compaction API call itself to exceed context limits (users frequently attach images in CCD sessions), so text markers are used instead.

PS: stripImagesFromMessages only takes effect in the streaming fallback, and the streaming fallback doesn't use the main conversation's KV cache at all, so it has no impact on main conversation KV cache hits.

7.5 Post-Compaction File Recovery

```
createPostCompactFileAttachments(
  preCompactReadFileState,  // All file reads tracked before compaction
  context,
  maxFiles: 5,              // POST_COMPACT_MAX_FILES_TO_RESTORE
  preservedMessages?,       // Preserved messages (avoid duplicate injection)
): AttachmentMessage[]
```

File Selection Strategy:

1. Extract all files from `readFileState` (sorted by most recent access time)
2. Exclude plan files (`getPlanFilePath()`) and all memory files
3. Exclude files that already have Read results in `preservedMessages` (avoid duplication)
4. Take the most recent 5

Token Budget Control:

```
POST_COMPACT_TOKEN_BUDGET = 50_000   // Max 50K tokens total
POST_COMPACT_MAX_TOKENS_PER_FILE = 5_000  // Max 5K per file
// When file exceeds 5K, truncate from the beginning with a marker
```

Token Savings from File Recovery: Without recovery, the model needs to re-read these files after each compaction. At an average of 500 tokens per Read and 5 files, each compaction saves approximately 2.5K tokens.

7.6 Skill Recovery

```
createSkillAttachmentIfNeeded(agentId?): AttachmentMessage | null

// Skill content is key information that persists across compactions
// Max 5K tokens per skill
// Total skill budget 25K tokens
// Sorted by invocation time (newest first), oldest discarded when budget exceeded
```

Why not clean sentSkillNames: Re-injecting the complete skill listing (~4K tokens) is pure cache creation with no actual benefit. Invoked skill content is preserved via `invoked_skills` attachment, and new skill dynamic additions are handled by `skillChangeDetector` .

PS:

- InvokedSkillInfo retains "invocation timestamp" and "which agent invoked it". Invoked skills retain the original version because after compaction, the skill file path remains unchanged but file content may have been modified by the user. Directly storing a snapshot of the content at compaction time ensures consistency.
- InvokedSkills tracks skills that have already been invoked; a skill is recorded immediately after it's invoked.
- invokedSkills stores the static content of a skill (file text + parameter substitution). Parameter substitution refers to the `$ARGUMENTS` placeholder being replaced with actual parameters when the skill is invoked.

7.7 PTL Retry Mechanism

PS:

- Uses the fixed value of 135K to trigger prompt-too-long judgment.

- `tokenGap` refers to when the compaction API request itself triggers PTL (Prompt Too Long), and the API error message carries information about how many tokens over the limit.

```
// When compaction request itself triggers prompt_too_long
truncateHeadForPTLRetry(messages, ptlResponse): Message[] | null

// Max 3 retries (MAX_PTL_RETRIES = 3)
// Retry strategy:
// 1. Parse tokenGap from API error
// 2. Group by API round, discard from the oldest group first (one conversation round is one group)
// 3. Discard until tokenGap is covered
// 4. Fallback: if tokenGap cannot be parsed, directly discard 20% of groups

// Retry marker: '[earlier conversation truncated for compaction retry]'
// If the first message is this marker, strip it first before grouping (avoid infinite loop)
```

7.8 Prefer Forked Sub-Agent Path During Compaction

```
// compact.ts - streamCompactSummary priority path
if (promptCacheSharingEnabled) {
  const result = await runForkedAgent({
    promptMessages: [summaryRequest],
    cacheSafeParams,        // Inherit parent session's cache parameters
    canUseTool: createCompactCanUseTool(),  // Disable tools
    querySource: 'compact',
    forkLabel: 'compact',
    maxTurns: 1,            // Only allow 1 turn
    skipCacheWrite: true,    // Don't write new cache (current context already cached)
    // Pass abortController for compaction context: user Esc can interrupt
  })
  // Extract summary text from result messages
}
```

Why pass parent session's cacheSafeParams: The compaction fork needs to use exactly the same cache key (system prompt + tools + message prefix) as the main session to hit the prompt cache already created by the main session. If the cache key doesn't match, the fork's cache hit rate is extremely low (about 98% miss rate for 3P users).

PS: The system prompt during compaction remains unchanged. -> But after stripping images and other context, shouldn't the KV cache also not hit?

```
所以子 agent 收到的完整请求是:
系统提示词 (继承) : "You are a helpful AI assistant..."
↓
工具定义 (继承) : Read, Edit, Bash, ...
↓
对话历史 (继承) : [所有压缩前的消息]
↓
用户消息 (摘要请求) : "请按以下格式总结上述对话: ..."
```

	Fork 路径	直接流式路径
系统提示词	**父进程的** (coding assistant)	**专用的** (summarizing)
能否复用 KV Cache	✅ 能（系统提示词一样）	❌ 不能（提示词变了）
摘要能力来源	user message 指令 + canUseTool 禁止	系统提示词直接定义

7.9 Direct Streaming Path (Fallback)

When the fork path fails or cache sharing is disabled, use direct streaming call:

```
// Tool list: only FileReadTool (+ ToolSearchTool if enabled)
// Key: do not set maxOutputTokens (avoid mismatch with parent session cache key)
// Alternative: set in options.maxOutputTokensOverride
```

PS:

- During compaction, directly call API in the current process and receive the summary via streaming.
- At this time there is a dedicated summary system prompt, different from the parent process. -> Essentially, KV cache cannot be hit either.

---

8. Session Memory

Session Memory is a **preventive** context management mechanism. During the conversation, it continuously maintains a Markdown note in the background, **extracting key information**. This allows compaction operations to replace the entire conversation history with "session notes + preserved tail messages" while retaining a large amount of context.

8.1 Core Difference from Compaction

	Full Compaction	**Session Memory**
Timing	When context pressure is too high	Continuously maintained during conversation
Method	Generate summary from scratch	**Incremental update** of existing notes
Content	Regenerated each compaction	Gradually accumulated
API Call	Required	Required (**Sub-Agent Update**)
Info Coverage	Depends on model summary quality	Incremental accumulation, more comprehensive

8.2 Trigger Conditions

```
// sessionMemoryUtils.ts
DEFAULT_SESSION_MEMORY_CONFIG = {
  minimumMessageTokensToInit: 10_000,    // Init: context at least 10K tokens
  minimumTokensBetweenUpdate: 5_000,     // Update: grow at least 5K tokens since last time
  toolCallsBetweenUpdates: 3,            // Update: at least 3 tool calls
}
```

Trigger Conditions (must **all be satisfied**):

```
shouldExtractMemory(messages: Message[]): boolean {
  // 1. Already initialized (minimumMessageTokensToInit satisfied)
  // 2. Token growth >= minimumTokensBetweenUpdate
  // 3. Tool call count >= toolCallsBetweenUpdates
```

```
  // 4. Last assistant message has no tool calls (natural conversation pause point)

  const shouldExtract =
    (hasMetTokenThreshold && hasMetToolCallThreshold) ||
    (hasMetTokenThreshold && !hasToolCallsInLastTurn)
}
```

Why require "no tool calls in last message": This ensures extraction happens at a natural conversation gap, not in the middle of tool calls, avoiding missing context.

8.3 Sub-Agent Execution Flow

```
Main conversation loop (after each response)
    ↓
postSamplingHook: extractSessionMemory
    ↓
Check trigger conditions (shouldExtractMemory)
    ↓
setupSessionMemoryFile()
  ├─ Create ~/.claude/sessions/<id>/session-memory.md (if not exists)
  ├─ Initialize with template
  └─ FileReadTool reads current content
    ↓
buildSessionMemoryUpdatePrompt(currentMemory, memoryPath)
  ├─ Load custom prompt.md or use default
  ├─ Analyze token size of each section
  └─ Generate warning if exceeded
    ↓
runForkedAgent({
  promptMessages: [userPrompt],
  canUseTool: createMemoryFileCanUseTool(memoryPath),  // Only allow Edit on this file
  querySource: 'session_memory',
  forkLabel: 'session_memory',
  maxTurns: 1,    // Usually completes in 1 turn (model directly Edits)
})
    ↓
Record token usage
Update lastSummarizedMessageId
```

PS: The session memory sub-agent reuses the main agent's system prompt, so KV can hit, and Claude supports multiple cache markers.

8.4 Session Memory Template Explained

```
# Session Title
_Short and distinctive 5-10 word title_
(Example: Optimize PostgreSQL query performance)

# Current State
_What is currently being worked on? Unfinished pending tasks. Next steps._
(This is the most critical section; look here first after compaction)

# Task specification
```

```
_What did the user ask to build? Design decisions and other context._

# Files and Functions
_What are the important files? Brief description of their content and relevance._
(includes file paths, function signatures, key logic)

# Workflow
_Common bash commands and their order? How to interpret outputs?_
(Deployment process, test commands, startup commands, etc.)

# Errors & Corrections
_What errors were encountered and how were they fixed? What did the user correct? Failed
methods?_
(includes error messages, fix steps, user feedback)

# Codebase and System Documentation
_What are the important system components? How do they work/compose?_

# Learnings
_What worked well? What didn't? What to avoid?_

# Key Results
_Specific outputs requested by the user (answers, tables, documents). Exact repetition._
(Preserve complete tables, code snippets, answers)

# Worklog
_Brief record of each step._
(Concise timeline for easy review)
```

PS: From early experience, the following sections can be added:

```
# Global Constraints & Preferences
_Behavioral boundaries and absolute must-not-do things_
(Examples: No direct main branch modification, output must strictly follow a specific style)

# Environment Context
Current execution task's specific system and external environment context
(includes database connection string format, specific port numbers, system and key
dependency versions)
```

8.5 Token Limits

```
// sessionMemory/prompts.ts
MAX_SECTION_LENGTH = 2000          // Max 2000 tokens per section
MAX_TOTAL_SESSION_MEMORY_TOKENS = 12000  // Max 12K tokens total for file

// When limits are exceeded:
// 1. Oversized sections: generate "section exceeds limit, please condense" warning
// 2. Total token limit exceeded: add "file exceeds 12K tokens, please condense all
sections" at prompt end
// During compaction, use truncateSessionMemoryForCompact():
```

```
//    Max 8000 characters per section
//    Truncate excess and add "content truncated" marker
```

PS:

Scenario 1: Sub-agent updating session memory

Let the sub-agent condense itself; the sub-agent's prompt has additional content at the end:

```
  Oversized sections to condense:
  - "# Errors & Corrections" is ~3500 tokens (limit: 2000)
  - "# Key results" is ~2800 tokens (limit: 2000)

  CRITICAL: The session memory file is currently ~14000 tokens,
  which exceeds the maximum of 12000 tokens.
```

Scenario 2: Injecting session memory into messages during compaction

Call truncateSessionMemoryForCompact → directly truncate via function, no sub-agent involved.

```
  Character limit per section = MAX_SECTION_LENGTH × 4 = 2000 × 4 = 8000 characters
                      ↑ roughTokenCountEstimation uses length/4
                      so character limit = 2000 tokens × 4 chars/token

  Processing:
    1. Split each section by '# '
    2. For each section, keep lines from the beginning
    3. Keep until character limit is reached
    4. Add truncation marker

  Original content (some section):
    # Errors & Corrections
    _Errors encountered and fixes..._
    - Error1: cause, fix method
    - Error2: cause, fix method
    - Error3: cause, fix method  ← exceeded limit
    - Error4: cause, fix method  ← truncated
    - Error5: cause, fix method  ← truncated

  After truncation:
    # Errors & Corrections
    _Errors encountered and fixes..._
    - Error1: cause, fix method
    - Error2: cause, fix method
    - Error3: cause, fix method
    [... content truncated; use Read on the skill path if you need the full text]
    ↑ Truncation marker, tells model it can re-read the skill file
```

8.6 Custom Template Support

```
// ~/.claude/session-memory/config/template.md  → Session memory template
// ~/.claude/session-memory/config/prompt.md    → Update prompt
// Custom prompt uses {{currentNotes}} and {{notesPath}} variables
```

8.7 Tool Usage Restrictions

```
createMemoryFileCanUseTool(memoryPath): CanUseToolFn
// Only allow: Edit tool, and only if file_path === memoryPath
// Deny: all other tools
// This is an extremely restricted environment, preventing sub-agent from making unexpected
operations
```

8.8 Mutual Exclusion with Main Agent

Mutual exclusion rules between session memory sub-agent and main agent:

- **Main agent writes**: Skip sub-agent's extraction this turn, advance cursor
- **Sub-agent writes**: Main agent won't write again
- **Both attempt to write simultaneously**: Impossible, as they are in different processes

8.9 Usage of Session Memory

The main agent does not proactively read this file. It **is only used during compaction**.

```
autoCompact / manual compact
    └── trySessionMemoryCompaction()
          ├── getSessionMemoryContent()         ← Read file directly
          ├── isSessionMemoryEmpty()           ← Check if has real content
          ├── calculateMessagesToKeepIndex()   ← Calculate preserved range based on
lastSummarizedMessageId
          └── createCompactionResultFromSessionMemory()
                ├── truncateSessionMemoryForCompact()   ← Truncate oversized content
                ├── getCompactUserSummaryMessage()       ← Wrap content as summary user
message
                └── buildPostCompactMessages()          ← Add to message history

  After compaction, session memory file content enters the main agent's
  messages list through the summary user message, indirectly becoming
  part of the conversation context
```

---

9. Session Memory Compaction (SM-Compact, Important)

Session Memory Compaction (SM-Compact) is an **alternative** to full compaction, using incrementally maintained session notes instead of generating summaries from scratch each time.

9.1 Comparison with Traditional Compaction

Dimension	Traditional Compaction	SM-Compact
Summary source	Generated from scratch each time	Incrementally maintained session notes
History handling	All replaced with summary	Selectively preserve tail
API calls	Required (call model to generate summary)	**Not required**
Content preserved	Only summary text	Summary + most recent 10K-40K tokens
Applicable to	General	Long multi-turn sessions

PS:
SM-Compact has higher priority than traditional compaction. When SM-Compact fails, it falls back to traditional compaction.

SM-Compact advantages:

```
  Traditional compactConversation requires each time:
  fork agent → call summarization API → wait for response → generate summary

  SM-Compact only requires:
  Directly read summary.md file → generate summary user message → return

  No API call, zero latency, zero cost
```

9.2 Preserved Message Calculation

```
// services/compact/sessionMemoryCompact.ts
calculateMessagesToKeepIndex(messages, lastSummarizedIndex): number

// Starting from lastSummarizedMessageId, expand backward:
// 1. Satisfy minTokens (default 10K)
// 2. Satisfy minTextBlockMessages (default 5)
// 3. Not exceed maxTokens (default 40K)
// 4. floor = last compaction boundary + 1 (don't cut inside compaction boundary)
```

`adjustIndexToPreserveAPIInvariants` **Protection**:

The API has two hard constraints that compaction must satisfy:

Constraint 1: Don't split tool_use/tool_result pairs

Constraint 2: Don't split streaming messages sharing message.id
PS: This is caused by API behavior in streaming mode. Specifically, in streaming mode, an assistant response arrives in segments. Therefore, as long as any block of a message.id is preserved, all blocks of that message.id must be preserved — because the API ultimately merges by message.id.

```
Claude's streaming emits separate assistant messages for each content block:
  Index N:   assistant(id=X), content: [thinking]      ← streaming chunk 1
  Index N+1: assistant(id=X), content: [tool_use: ID]  ← streaming chunk 2
  Index N+2: user, content: [tool_result: ID]

If startIndex = N+1 (only keep tool_use, discard thinking):
  → When normalizeMessagesForAPI merges by message.id
  → thinking block is lost
  → Model loses important reasoning process

Correct approach:
  → Detect that N+1 shares message.id with N
  → Adjust startIndex to N (include complete streaming group)
```

9.3 Feature Usage Determination

Trigger condition: `effective context window × 90%` (traditional compaction trigger threshold is `effective context window × 93.5%`)

Multi-layer progressive check for whether the `Session Memory Compact` feature can be used:

```
shouldUseSessionMemoryCompaction(): boolean
// 1. `tengu_session_memory` feature flag = true; main switch: whether session memory itself is enabled
// 2. `tengu_sm_compact` feature flag = true; sub-switch: whether session memory driven compaction is enabled
// 3. Check environment variables (feature switches): ENABLE_CLAUDE_CODE_SM_COMPACT (force enable)
//    or DISABLE_CLAUDE_CODE_SM_COMPACT (force disable)
// 4. Session memory file exists and is non-empty (isSessionMemoryEmpty())
```

9.4 Execution Flow

```
trySessionMemoryCompaction(messages, agentId, autoCompactThreshold)
    ↓
1. Wait for any ongoing session memory extraction to complete (15s timeout)
    ↓
2. Get session memory file content
    ↓
3. Locate lastSummarizedMessageId (try to recover if missing)
    ↓
4. Calculate message range to preserve (calculateMessagesToKeepIndex)
    ↓
5. Filter out old compaction boundary messages
    ↓
6. Execute SessionStart Hooks (reload CLAUDE.md)
    ↓
7. Create compaction result
  ├─ createCompactBoundaryMessage()
  ├─ Convert session memory content to summary message
  ├─ truncateSessionMemoryForCompact() (ensure within budget)
  └─ Attach plans etc.
    ↓
8. Check postCompactTokenCount < autoCompactThreshold
    ↓ Pass → return result
    ↓ Failed → return null, degrade to traditional compaction
```

9.5 Compaction Boundary Metadata

```
// compact.ts
annotateBoundaryWithPreservedSegment(
  boundary,
  anchorUuid,   // Anchor: last summary message
  messagesToKeep,
): SystemCompactBoundaryMessage

// preservedSegment tells session storage how to restore chain structure:
// Points to original messages in physical storage, used to tell session storage
```

```
// how to rebuild the relationship before and after compaction

// headUuid → first of preserved segment; UUID of the oldest discarded message
// anchorUuid → summary message (mount point); UUID of the new summary message generated by summarizer
// tailUuid → last of preserved segment; UUID of the last message in preserved segment
```

---

10. Auto-Compact

Auto-compaction is a compaction mechanism that automatically triggers when the context approaches a threshold, without requiring the user to manually execute `/compact`.

PS: There is also a reactive compact, but no specific implementation code is provided.

10.1 Threshold System Explained

```
┌──────────────────────────────────────────────────────────────────────┐
│                        Effective Context Window                      │
│                                                                      │
│  0%        warning          autocompact        blocking     100%    │
│   ├──────────┤├──────────────┤├────────────────┤├──────────┤        │
│   0         180K           187K              197K        200K      │
│             ↑              ↑                ↑                      │
│      isAboveWarning    shouldAutoCompact  isAtBlocking             │
│       Threshold           → Auto-compact    Limit → Block input    │
│                                                                      │
│  WARNING_BUFFER = 20K    AUTO_BUFFER = 13K  MANUAL_BUFFER = 3K     │
└──────────────────────────────────────────────────────────────────────┘
```

10.2 Warning States

```
calculateTokenWarningState(tokenUsage, model): {
  percentLeft: number,                      // Remaining space percentage
  isAboveWarningThreshold: boolean,          // Issue warning (yellow)
  isAboveErrorThreshold: boolean,            // Issue error (red)
  isAboveAutoCompactThreshold: boolean,     // Trigger auto-compaction
  isAtBlockingLimit: boolean,                // Block input
}
```

10.3 Compaction Priority

```
autoCompactIfNeeded(messages, toolUseContext, cacheSafeParams, ...): {
  wasCompacted: boolean,
  compactionResult?: CompactionResult,
  consecutiveFailures?: number,
}

// Execution order:
// 1. Check circuit breaker (consecutive failures >= 3 → return)
// 2. shouldAutoCompact() → check threshold
// 3. Try SM-Compact (trySessionMemoryCompaction)
//    → Success → return
```

```
// 4. Execute traditional compaction (compactConversation)
//    → Success → reset circuit breaker consecutiveFailures = 0
//    → Failure → consecutiveFailures++
```

10.4 Circuit Breaker Explained

Circuit breaks after three consecutive failures.

```
const MAX_CONSECUTIVE_AUTOCOMPACT_FAILURES = 3

// Data background (2026-03-10):
// 1,279 sessions had 50+ consecutive failures (max 3,272 times)
// ~250K wasted API calls per day

// When failing consecutively:
// - Skip futile compaction attempts
// - User receives "conversation full" error, must handle manually (/clear or /compact with
options)
// - Next new input re-checks threshold
```

10.5 Disable Control Matrix

Control Method	Effect	Priority
`DISABLE_COMPACT=1`	Disable all compaction	Strongest
`DISABLE_AUTO_COMPACT=1`	Only disable auto-compaction, keep `/compact`	Strong
`userConfig.autoCompactEnabled=false`	User config	Medium
`CONTEXT_COLLAPSE` feature	Collapse system takes over, disable auto-compact	Medium
`REACTIVE_COMPACT` feature	Disable proactive compaction, rely on 413 response	Medium

10.6 Special Nature of Marble Origami

When `CONTEXT_COLLAPSE` is enabled, marble_origami (ctx-agent)'s auto-compaction is disabled:

- If ctx-agent context blows up, auto-compaction triggers
- `runPostCompactCleanup` calls `resetContextCollapse()`
- This would break the **main thread**'s committed log (shared module state)

Solution: When `CONTEXT_COLLAPSE` is enabled, ctx-agent disables auto-compaction.

PS:

- marble_origami is a ctx-agent (context collapse agent), running in sub-agent/fork. It shares the same process module-level state with the main thread.
- marble_origami is a ctx-agent with shared process state. If its auto-compact succeeds, it triggers `resetContextCollapse()` — even with the isMainThreadCompact check, the existence of shared state itself is unsafe. So at the very source of shouldAutoCompact, marble_origami is excluded. The ctx-agent's context explosion issue is handled by the Context Collapse system itself, not going through the auto-compact path.

---

11. Context Collapse (Experimental Feature, all content is speculative)

Context Collapse (`CONTEXT_COLLAPSE` feature flag) is an **experimental** context management mechanism controlled by a feature gate.

PS:

- Context Collapse is an already-implemented, currently in internal gradual testing experimental feature, not yet officially released. It has neither documentation nor stable product naming externally.
- Because the core service source code (services/contextCollapse/) is not included in restored-src/, the complete logic is only inferred through cross-analysis of 14 reference points.

11.1 Core Difference from Compaction

PS: Collapse is implemented in code, without calling AI.

```
Compaction:
  Discard old messages → replace with summary
  Information loss: medium (controlled by prompt quality)
  Latency: high (requires calling AI model to generate summary)
  Precision: low-medium (summary quality unstable)

Context Collapse:
  Selective discard → precise control over preserved content
  Information loss: precisely controllable
  Latency: low (no AI call needed)
  Precision: high (precise proportional discard)
```

11.2 Design Principles

Similar to operating system memory management paging strategy:

- **90% threshold**: Start commit operation — write intermediate results to disk
- **95% threshold**: Block new input — wait for commit to complete
- **Preservation strategy**: Keep the most recent N% of messages, ensuring working context is not lost

11.3 Mutual Exclusion with Auto-Compact

Priority order: **Context Collapse > Session Memory Compact > Auto-Compact**

```
When CONTEXT_COLLAPSE is enabled:
  - Auto-Compact is completely suppressed (avoid competition)
  - 90% commit threshold < 93.5% auto-compact threshold
  - Collapse system takes over before auto-compact
  - 95% blocking threshold > 93.5% auto-compact threshold

When CONTEXT_COLLAPSE is disabled:
  - Auto-Compact operates normally
  - Depends on summarization to manage context
```

11.4 Implementation Location

```
// Dynamic import (avoid circular dependency):
if (feature('CONTEXT_COLLAPSE')) {
  const {
```

```
    isContextCollapseEnabled,
    applyCollapsesIfNeeded,
    recoverFromOverflow,
    isWithheldPromptTooLong,
    resetContextCollapse,
  } = require('../services/contextCollapse/index.js')
}
```

Why dynamic import: This module is in the `services/contextCollapse/` directory and was not included in the restored source file list in the source map restoration. This indicates `contextCollapse` is a compiled standalone module, dynamically loaded via `require()`.

11.5 Objects Being Collapsed

Operations are performed on **local context** for each message span, so the idea is still collapsing. Specifically:

```
  user message → assistant message → user message → ... → assistant message
   ↑                                                            ↑
   firstArchivedUuid (span start)                          lastArchivedUuid (span end)
```

The product of collapsing: each collapsed span is replaced with a metadata placeholder:

```
<collapsed id="16-char-collapseId">Summary text</collapsed>
```

11.6 Two-Phase Collapse Algorithm

Context Collapse is **preventive and progressive**.

Phase 1 — Staging (Pre-collapse)

Trigger: 90% effective context window
Effective window tokens ≥ effective window × 90%

At this point:

1. ctx-agent spawn (sub-process, codename "marble_origami") is started
2. ctx-agent analyzes all message spans from "protected tail boundary" to current position
3. **Generates summary for each span**: { startUuid, endUuid, summary, risk, stagedAt }
4. Result is written to staged queue (in memory)
5. Writes a marble-origami-snapshot entry to transcript (persist staged queue state)
6. Staged spans are not immediately deleted from API context, just waiting in queue

> Significance of staging: Prepare summaries in advance, don't wait until overflow begins.

Phase 2 — Commit (Formal Collapse)

Trigger conditions have two paths:

Path A (Proactive) — 95% blocking trigger
Effective window tokens ≥ effective window × 95%
Before the next spawn request is sent, check if staged:

- If staged exists → directly commit (use existing summary, no need to call model again)
- If not staged → immediately generate summary then commit

Path B (Reactive) — 413 overflow recovery

API returns 413 (prompt too long)

Call recoverFromOverflow():

1. Commit all spans in staged queue (drain)
2. Retry API request with new context after commit
3. If retry again returns 413 → fall back to reactiveCompact

11.7 Relationship with Other Mechanisms

```
用户输入 + 完整历史消息
        │
        ▼
  ① microcompact
        │ tool_use_id 去重 + 移除 cache metadata
        │ (只处理格式，不减少 token 数)
        ▼
  ② snipCompact(可选，HISTORY_SNIP 开关)
        │ 截断超长 tool result
        │ tokensFreed 会传给 autocompact 的阈值计算
        ▼
  ③ Context Collapse — applyCollapsesIfNeeded()
        │ 核心逻辑：staging + commit
        │ 仅当 feature('CONTEXT_COLLAPSE') 开启
        ▼
  ④ autocompact
        │ ⚠ 当 Context Collapse 开启时 → 完全跳过 (shouldAutoCompact = false)
        │ 当 Context Collapse 关闭时 → 正常触发 (~93% 阈值)
        ▼
  ⑤ API调用 (callModel)
```

机制	触发时机	效果	持久化	优先级
microcompact	始终（每轮）	格式去重，不删内容	否	最先
snipCompact	HISTORY_SNIP 开启时	截断长 tool result	boundary 消息	其次
Context Collapse	90% staging / 95% commit	span → placeholder	commit + snapshot 条目	第三
autocompact	~93%，CC 关闭时	批量 summary	summary 消息	CC 开启时跳过
reactiveCompact	CC 兜底后 / RC 单独启用	全量 summary	summary 消息	最后兜底

```
isContextCollapseEnabled() ?
│
├── FALSE → autocompact 正常检查 (~93%)
│              └── reactiveCompact 作为 413 fallback
│
└── TRUE
     │
     ├── autocompact 完全跳过 (shouldAutoCompact = false)
     │
     ├── API 调用前：applyCollapsesIfNeeded()
     │    │
     │    ├── tokens < 90%  → no-op
     │    ├── tokens ≥ 90%  → spawn ctx-agent, stage spans
     │    └── tokens ≥ 95%  → commit staged spans
     │
     └── API 返回 413:
          │
          ├── recoverFromOverflow()
          │     └── drain → retry → 还 413?
          │           │
          │           └── reactiveCompact (兜底)
          │
          └── 如果 reactiveCompact 也无 → 暴露错误
```

12. Context Analysis

12.1 Use Cases

`analyzeContext()` scans the message array and generates detailed statistics on token consumption, used for:

1. **Telemetry reporting**: Report detailed dimensions of `tengu_compact` event via Statsig
2. **Diagnostic analysis**: Identify optimizable waste points (e.g., duplicate file reads)

PS:

- analyzeContextUsage() is not just for token consumption statistics; it is a complete context audit system — responsible for perception, classification, quantification, and driving actionable optimization suggestions.
- `tengu` is Anthropic's internal project code prefix, found in 1051 places across 250 files. In Japanese folklore, Tengu is a highly skilled spirit. This prefix suggests the importance of this project to Anthropic internally.
- The "identifying optimizable waste points" feature is entirely based on regular expression statistics + hardcoded heuristic rules, without calling any LLM.

12.2 Statistical Dimensions

```
type TokenStats = {
  toolRequests: Map<string, number>       // Token consumption by tool request
  toolResults: Map<string, number>        // Token consumption by tool result
  humanMessages: number                    // Total human message tokens
  assistantMessages: number               // Total assistant message tokens
  localCommandOutputs: number             // Bash command output tokens
  other: number                           // Other (thinking, attachments, etc.)
  attachments: Map<string, number>        // Count by attachment type
  duplicateFileReads: Map<string, {        // Duplicate file reads
    count: number
    tokens: number
  }>
  total: number                           // Total tokens
}
```

12.3 Duplicate Read Detection Algorithm

```
// For multiple reads of the same file, count only 1 time
// Duplicate tokens = avgTokensPerRead × (count - 1)

fileReadStats.forEach((data, path) => {
  if (data.count > 1) {
    const avgTokensPerRead = Math.floor(data.totalTokens / data.count)
    const duplicateTokens = avgTokensPerRead * (data.count - 1)
    stats.duplicateFileReads.set(path, { count, tokens: duplicateTokens })
  }
})
```

Practical Application: If a file is read 10 times at 1K tokens each, then `duplicate_read_tokens` = 9K. This means after compaction, the model doesn't need to re-read the file, saving significant tokens.

12.4 Statsig Metric Mapping

```
tokenStatsToStatsigMetrics(stats): Record<string, number>

// Generates 40+ metrics, including:
// - Token consumption and percentage by tool
// - Duplicate read tokens and file count
// - Token distribution by message type
// - Count by attachment type
```

固定 Token 指标（5个）

指标名	含义
total_tokens	消息总 token 数（不含 system prompt / tools）
human_message_tokens	用户消息的 token 数（含图片被 strip 成 [image] 后的 text）
assistant_message_tokens	助手消息的 token 数（含 thinking、tool_use block 的文本表示）
local_command_output_tokens	本地命令输出 token（local-command-stdout 标记的内容，通常是 BashTool 执行结果的文本）
other_tokens	其他所有块类型的 token（thinking、search_result、document、compaction 等）

附件计数（n个，每种类型一个）

指标名	含义
attachment_${type}_count	该类型附件出现的次数

常见类型：
- attachment_file_count — 文件附件（createFileAttachment / post-compact 恢复的文件）
- attachment_skill_discovery_count — 技能发现信号
- attachment_skill_listing_count — 技能列表
- attachment_command_permissions_count — 命令权限附件
- attachment_invoked_skills_count — 被调用的 skill 附件
- attachment_plan_mode_count — plan mode 附件

工具请求 Token（n×2个，每工具）

指标名	含义
tool_request_${tool}_tokens	该工具被调用时，tool_use block 本身的 token 数（包含工具名、参数名/值的序列化文本）
tool_result_${tool}_tokens	该工具返回结果，tool_result block 的 token 数

```
tool_request_Read_tokens      → Read tool 调用时发出的参数（file_path 等）
tool_result_Read_tokens       → Read tool 返回的文件内容 token
tool_request_Bash_tokens      → Bash 调用参数
tool_result_Bash_tokens       → Bash 执行结果输出 token
tool_request_Edit_tokens      → Edit 参数
tool_result_Edit_tokens       → Edit 结果
...
```

重复文件读取（2个）

指标名	含义
duplicate_read_tokens	所有被重复读取的文件的浪费 token 总数。计算方式：avg_tokens_per_read × (count - 1)（每次重复读取浪费 1 倍平均大小）
duplicate_read_file_count	出现了重复读取的文件种类数（有多少个不同文件被读了多次）

注意：duplicate_read_tokens 是**浪费量的估算**，不是实际消耗。

百分比指标（6个固定 + n×2个动态）

指标名	含义
human_message_percent	用户消息 token 占总消息 token 的百分比
assistant_message_percent	助手消息 token 占比
local_command_output_percent	本地命令输出 token 占比
tool_request_percent	所有 tool_request block 总 token 占比
tool_result_percent	所有 tool_result block 总 token 占比
duplicate_read_percent	重复读取浪费 token 占总 token 的百分比
tool_request_${tool}_percent	该工具的 request token 占总 token 的百分比
tool_result_${tool}_percent	该工具的 result token 占总 token 的百分比

百分比都是四舍五入到整数。

categories overview

```
  ContextData.categories = [
    { name: 'System prompt',        tokens: N },   // System prompt section details
    { name: 'System tools',        tokens: N },   // Built-in tool schemas (ANT users can
see each tool's details)
```

```
    { name: 'MCP tools',           tokens: N },   // MCP server tools (after deduplication)
    { name: 'MCP tools (deferred)', tokens: N, isDeferred: true },  // Loaded but not
consuming
    { name: 'System tools (deferred)', tokens: N, isDeferred: true },
    { name: 'Custom agents',       tokens: N },
    { name: 'Memory files',        tokens: N },   // Each CLAUDE.md file details
    { name: 'Skills',              tokens: N },   // Skill frontmatter (excluding full
content)
    { name: 'Messages',            tokens: N },   // Conversation history (after
microcompact)
    { name: 'Autocompact buffer',   tokens: N },   // Safety space reserved for autocompact
(skipped when CC enabled)
    { name: 'Free space',          tokens: N },
  ]
```

Key: Categories with `isDeferred: true` are not counted in actualUsage — they represent parts that are "loaded but only consume tokens on demand."

Deferred Tool:

- Only tell the model the tool name, not the full schema, load on demand
- Important: User-configured MCP tools are the primary target: **MCP tools are deferred by default** (isMcp === true takes that branch), while built-in tools are only deferred if explicitly marked shouldDefer (some built-in tools are prohibited from being deferred).
- Normally, each API request includes all tools' complete schemas (name + description + parameters) in the request header. The Deferred mechanism changes this:

```
  Normal tools (non-deferred)
    → API request contains complete schema (token consumption is immediate)

  Deferred tools
    → API request has only tool name (minimal tokens)
    → When model wants to call it → first call ToolSearchTool to pull full schema → then
execute
```

The idea is similar to progressive disclosure of skills. The code even directly states this:

```
  // loadSkillsDir.ts:97
  /**
   * Estimates token count for a skill based on frontmatter only
   * (name, description, whenToUse) since full content is
   * only loaded on invocation.
   */
  export function estimateSkillFrontmatterTokens(skill: Command): number
```

Skill 的渐进式披露（对比 Deferred Tool）

	Skill	Deferred Tool
披露时机	首次被调用时	首次被调用时
初始加载	name + description + whenToUse（frontmatter）	只有工具名
完整内容	--- 分隔符后的 markdown 全文	完整 JSONSchema
触发方式	模型调用 SkillTool	模型调用 ToolSearchTool
加载后	指令直接执行，不重复调用	schema 进入可用工具集

统一的设计模式：按需加载（Lazy Load）

```
Skills:
  初始：frontmatter token 数 → 模型感知到 N 个 skill 存在
  调用：SkillTool → 完整 markdown 加载 → 执行指令

Deferred Tools:
  初始：工具名字典 → 模型感知到 N 个工具存在
  调用：ToolSearchTool → 完整 schema 加载 → 可调用

Deferred MCP Tools（always deferred）：
  初始：mcp__server__toolname 只有名字
  调用：ToolSearchTool → 完整 schema → 执行
```

共同解决的问题

```
问题：工具/skill 全集太大，全部塞进上下文太贵
解法：不塞全集，只塞"目录"（名字 + 简介）

代价：第一次调用有额外延迟（需要拉取完整内容）
收益：未使用的部分完全不消耗 token
```

12.5 How Detectable Waste Points Are Identified

Waste detection consists of two systems:

检查函数	检测条件	优化建议
checkNearCapacity	percentage ≥ 80%	"建议现在手动 /compact，否则 autocompact 会自动丢失消息"
checkLargeToolResults	某 tool 的 call+result > 15% 上下文 或 > 10k tokens	"工具 X 的结果太大，考虑分流或截断"
checkReadResultBloat	Read tool results > 5% 上下文	"Read 结果太冗余，考虑只读必要的部分"
checkMemoryBloat	CLAUDE.md > 5% 上下文 或 > 5k tokens	"CLAUDE.md 太大，考虑精简"
checkAutoCompactDisabled	autocompact 关闭 且上下文 > 80%	"autocompact 关闭，手动 /compact 来释放空间"

System A: generateContextSuggestions() — User-facing suggestions

System B: checkContextWarnings() — Warnings for doctor command

检查函数	检测条件	警告
checkClaudeMdFiles	任意 CLAUDE.md 文件 > 40k chars	列出每个大文件及字符数
checkAgentDescriptions	agent 描述 token > 阈值	列出每个 agent 的估算 token
checkMcpTools	MCP tools 总 token > 25k	按 server 分组列出 token 分布
checkUnreachableRules	权限规则被其他规则遮蔽	列出每个失效规则 + 修复建议

12.6 Summary

User-facing UI layer
/context command ← ContextData renders
ContextVisualization ← gridRows + categories render
TokenWarning ← percentage triggers warning

Suggestion layer
contextSuggestions ← Proactive suggestions (what can be optimized)
doctorContextWarnings ← Warnings (config/scale issues)

Statistics layer
analyzeContextUsage ← Bucket counting, API/client dual precision:

When API returns actual usage, use server numbers; otherwise use client estimate
← Deferred mechanism (saves tokens)
← Per-tool granularity breakdown, statistics on "which tool costs most", show message distribution by tool type
← Analyzed after microcompact, ensuring statistics reflect actual tokens the model sees

---

13. Post-Compaction Cleanup

`runPostCompactCleanup()` executes after each compaction (automatic/manual) to clean up state polluted by compaction operations.

13.1 Cleanup Content Explained

```
runPostCompactCleanup(querySource?: QuerySource): void

// Cleanup only executed for main thread compaction:
resetMicrocompactState()              // Reset microcompact state (tool registration, cache
edit queue)
getUserContext.cache.clear()          // Clear user context cache (CLAUDE.md content)
resetGetMemoryFilesCache()            // Clear memory file discovery cache
clearSystemPromptSections()           // Clear system prompt sections cache
clearClassifierApprovals()            // Clear classifier approvals (avoid approvals being
forgotten)
clearSpeculativeChecks()              // Clear speculative check state
clearBetaTracingState()               // Clear beta tracing state
clearSessionMessagesCache()           // Clear session messages cache
sweepFileContentCache()              // Clear post-attribution file cache
(COMMIT_ATTRIBUTION)

// Context Collapse (main thread only):
if (feature('CONTEXT_COLLAPSE') && isMainThreadCompact) {
  resetContextCollapse()              // Reset context collapse state
}
```

13.2 Key Items NOT Cleared

```
// The following are intentionally NOT cleared (persist across compactions):
// 1. invoked_skills → needed by createSkillAttachmentIfNeeded
// 2. sentSkillNames → re-injecting complete listing is pure waste
```

PS:
invoked_skills records skills that have already been executed. The skill currently being executed is not yet in invoked_skills. So "partial execution" is not an issue.
Then what does it matter if the model knows which skills have been used??
The LLM explanation is: preserve actual content of skills that have proven to be valuable. But this explanation seems to have a loophole... skills that have been executed once don't necessarily need to be executed again later.

13.3 Main Thread vs. Sub-Agent Distinction

```
// Sub-agents (agent:*, sdk) also call functions in compact.ts
// But their compaction should not clear main thread's cache
const isMainThreadCompact =
  querySource === undefined ||
  querySource.startsWith('repl_main_thread') ||
  querySource === 'sdk'
```

Sub-agent post-compaction cleanup:

Summary:
After sub-agent compaction:
✅ resetMicrocompactState() — clean up its own (isolated)
✅ contentReplacementState reset — clean up its own (isolated)
❌ runPostCompactCleanup() — not called (won't mistakenly clear main thread shared state)

```
压缩发生在主线程？
  ├── 是 → 清理所有状态（microcompact + 共享状态）
  └── 否（子 agent）→ 不调用
```

因为下面这些是**主线程共享的模块级状态**，子 agent 乱清会破坏主线程：

状态	为什么子 agent 不能清
resetContextCollapse()	CC commit log 是主线程的，清了主线程的折叠全丢了
clearSessionMessagesCache()	主线程的 transcript 缓存，清了 UI 会看到 stale 数据
getUserContext.cache.clear()	CLAUDE.md 是主线程的，子 agent 清理等于帮倒忙
clearSystemPromptSections()	system prompt sections 是主线程的渲染状态
sweepFileContentCache()	归因文件缓存是主线程的

而下面这些是**子 agent 自己隔离的**，所以子 agent 自己清：

状态	为什么子 agent 可以自己清
resetMicrocompactState()	子 agent 有自己的工具注册表和缓存编辑队列，不清会内存泄漏
contentReplacementState	子 agent 有自己的替换状态映射，与主线程隔离

After main thread compaction:
✅ runPostCompactCleanup() — all cleanup items execute

Core principle: Whoever compacts cleans their own isolated state; whoever calls runPostCompactCleanup cleans shared state. Sub-agent compaction is isolated; it only manages itself.

---

14. Context Loading and Caching

14.1 System Context (`context.ts`)

PS: Is `` the system prompt itself, or extra information injected into the system prompt?

```
注入后的 system prompt 结构

You are Claude Code...

You have access to tools: Read, Edit, Bash...

[环境信息 - systemContext]
Current branch: main
Status: (clean)
Recent commits: a8a678c init...

[用户配置 - userContext]
Today's date is 2026-04-02.

# CLAUDE.md 内容（从项目根目录加载）

[对话历史 - messages]
...
```

Git Status

```
getGitStatus(): Promise<string | null>
// Contains:
// - Current branch
// - Main branch
// - Git username
// - git status --short (max 2000 characters, truncated if exceeded)
// - Most recent 5 commits

// Caching: valid for entire session (memoized)
// Skip conditions:
  // 1. NODE_ENV=test (avoid test loops)
  // 2. Non-git directory (getIsGit() === false)
  // 3. CLAUDE_CODE_REMOTE environment variable (CCR mode, skip to save cost)
  // 4. Git instructions disabled (shouldIncludeGitInstructions())
```

User Context

```
getUserContext(): Promise<{ claudeMd, currentDate }>
// claudeMd: content loaded from all CLAUDE.md and memory files
// currentDate: current ISO date

// Caching: valid for entire session (memoized)
// Disable conditions:
  // 1. CLAUDE_CODE_DISABLE_CLAUDE_MDS=1 (hard disable)
  // 2. --bare mode with no explicit --add-dir (skip auto-discovery)
```

Cache Invalidation

```
// When context needs to be reloaded (e.g., user modified CLAUDE.md):
setSystemPromptInjection(value)  // Set cache breaker
// → getUserContext.cache.clear()
// → getSystemContext.cache.clear()
// → next call reloads
```

PS: CLAUDE.md loads at three moments:

- Moment 1: At session start
- Moment 2: After compaction (automatic reload)
- Moment 3: User modified CLAUDE.md (incremental update)

14.2 Session Storage (`utils/sessionStorage.ts`)

Message history is stored on disk in JSONL format, supporting chain-based recovery of compaction boundaries:

```
~/.claude/sessions/<session-id>/
├── messages.jsonl           ← Main message history
├── metadata.jsonl          ← Metadata (title, tags)
└── ...
```

Chain recovery: Each message records a parentUuid pointing to its parent message when stored. Recovery follows parentUuid from the newest message backward, forming a chain.

---

15. Persistent Memory Extraction

`extractMemories.ts` runs at the end of each complete query loop, writing key information to persistent storage.

15.1 Difference from Session Memory

	Session Memory	**Persistent Memory**
Location	`~/.claude/sessions/<id>/session-memory.md`	`~/.claude/projects/<path>/memory/`
Lifecycle	Current session	**Cross-session**
Content	Key work records for current session	Useful cross-session knowledge
Trigger	Context growth reaching threshold	End of each conversation turn (automatic)
Tool restrictions	Only Edit session memory file	Edit/Write in memory directory

"Each complete query loop" means: whenever the main agent finishes a conversation turn (model gives final response, no tool calls), stop hooks trigger extractMemories.

15.2 Flow

```
extractMemories executes
    ↓
1. Check if main agent has already written to memory file
   → Wrote → skip, advance cursor
   → Hasn't written → continue
    ↓
2. Throttle check
   → Should not execute this turn → return
   → Should execute → continue
    ↓
3. Scan existing files in memory directory
    ↓
4. Fork sub-agent
   → Read memory directory
   → Based on conversation delta (new messages after lastMemoryMessageUuid)
   → Generate update prompt
   → Use Edit/Write tools to write new memory files
    ↓
5. Update cursor lastMemoryMessageUuid
```

15.3 Storage Structure and Content

```
~/.claude/projects/<path>/memory/
├── MEMORY.md              ← Index file (links to all topics)
├── auth-implementation.md ← Topic file 1
├── database-schema.md     ← Topic file 2
└── ...
```

Memory Types: Four Types

1. user — User profile

```
<name>user</name>
- Content saved: user's role, goals, responsibilities, knowledge background
- When to save: when learning user's role preferences, responsibilities, or knowledge
- How to use: adjust style and depth based on user profile when answering questions
- Example:
  user: "I'm a data scientist investigating what logging we have in place"
  → Save: user is a data scientist, currently focused on observability/logging
```

2. feedback — Feedback preferences

```
<name>feedback</name>
- Content saved: guidance on working methods given by user (what to avoid, what to keep)
- When to save: when user corrects ("no not that") or confirms ("yes exactly")
- Structure: rule itself → Why → How to apply
- Example:
  user: "don't mock the database in these tests"
  → Save: integration tests must hit a real database, not mocks.
      Reason: prior incident where mock/prod divergence masked a broken migration
```

3. project — Project state

```
<name>project</name>
- Content saved: ongoing projects, goals, bugs, incidents
- When to save: when learning who is doing what, why, or by when
- Structure: fact → Why → How to apply
- Note: Convert relative dates to absolute dates ("Thursday" → "2026-03-05")
- Example:
  user: "we're freezing non-critical merges after Thursday — mobile team is cutting a
release branch"
  → Save: merge freeze begins 2026-03-05 for mobile release cut.
      Flag any non-critical PR work scheduled after that date
```

4. reference — External system references

```
<name>reference</name>
- Content saved: locations of information in external systems
- When to save: when learning about external resources and their purpose
- Example:
  user: "check the Linear project 'INGEST' if you want context on these tickets"
  → Save: pipeline bugs are tracked in Linear project 'INGEST'
```

Explicitly not saved content

```
- Code patterns, conventions, architecture, file paths → can be derived from reading code
- Git history, who-changed-what → git blame is authoritative
- Debugging solutions → fix is in the code; commit message has context
```

```
  - Anything already documented in CLAUDE.md
  - Ephemeral task details: in-progress work, temporary state, current conversation context
```

Meaning of "ephemeral task" in the memory system:

- In-progress work
- Temporary state
- Current conversation context

Write Format

Each memory is written to its own file with frontmatter:

```
  ---
  name: Memory name
  description: One-line description
  type: user | feedback | project | reference
  ---

  Memory body content...

  Index file MEMORY.md records pointers:
  - [Memory title](filename.md) — one-line hook
  - [Memory title2](filename2.md) — one-line hook
```

15.4 Sub-Agent Execution

```
// Tool restrictions: createAutoMemCanUseTool(memoryDir)
allow: Read, Grep, Glob, read-only Bash
allow: Edit, Write within memoryDir
deny: all other operations

// Throttling control:
// Execute once every N qualified turns
// getFeatureValue('tengu_bramble_lintel', 1)
// Default: execute every turn

// Max turns: 5 (prevent validation rabbit hole)
// skipTranscript: true (don't write to transcript, avoid race conditions)
```

PS:
extractMemories sub-agent doesn't have a dedicated system prompt — it uses the parent agent's system prompt (coding assistant). Task instructions are passed via user message:

```
You are now running as a memory extraction sub-agent.
Analyze the most recent ~{N} messages above and use them to update your persistent memory
system.

Available tools: Read, Grep, Glob, read-only Bash, and Edit/Write (only within memory
directory paths).
```

```
You must only use content from the most recent ~{N} messages. Don't waste turns on further
investigation or verification.

## Memory Types
[Detailed definitions of user / feedback / project / reference...]

## Content NOT to Save in Memory
[Code patterns / git history / content already in CLAUDE.md...]

## How to Save Memory
1. Read existing memory files first to avoid duplication
2. Write to separate files with frontmatter format
3. Update MEMORY.md index (each entry max 150 characters)
```

15.5 Mutual Exclusion Protection

```
// If the main agent wrote to the memory file this turn:
hasMemoryWritesSince(messages, lastMemoryMessageUuid)
// → Skip extraction, advance cursor
// → Mutual exclusion to avoid conflicts
```

15.6 Wait on Exit

```
// Before CLI is exited by user, drainPendingExtraction()
// Wait up to 60 seconds for extraction to complete
// Prevent fork process from being killed during shutdown
```

---

16. History Management

`history.ts` manages the REPL's prompt history (used for ↑ arrow recall and Ctrl+R search).

REPL: Read-Eval-Print Loop, i.e., Claude Code CLI's interactive terminal interface

16.1 Key Constants

```
MAX_HISTORY_ITEMS = 100          // Max 100 items
MAX_PASTED_CONTENT_LENGTH = 1024 // Max pasted content length
```

16.2 Main Functions

```
addToHistory(entry)                        // Add to history
getHistory()                               // Get current project history
getTimestampedHistory()                    // Version with timestamps (for picker)
removeLastFromHistory()                   // Undo last addition
expandPastedTextRefs()                     // Resolve pasted references
```

---

17. Message Normalization and Boundary Management

17.1 Message Normalization (`normalizeMessagesForAPI`)

This is the core function for converting internal message format to API request format:

```
normalizeMessagesForAPI(messages, tools): (UserMessage | AssistantMessage)[]
```

PS: Converts session structured content into LLM context.

Execution Steps:

1. **Reorder attachments**: `reorderAttachmentsForAPI()` bubbles attachments to appropriate positions
2. **Filter virtual messages**:

```
// The following types are filtered out, not entering API requests:
- progress messages
- system messages (except local_command)
- synthetic API error messages
```

3. **Process local commands**: `local_command` system messages are converted to user messages (merged into the previous user message)
4. **Process user messages**:
   - Merge consecutive user messages (Bedrock doesn't support consecutive user messages)
   - Strip tool_reference blocks (when tool search is disabled)
   - Strip blocks corresponding to failed attachments (PDF too long/password protected/invalid)
   - Handle text completions in tool_result (avoid stop sequence issues)
5. **Validate message order**: Ensure the first message has role='user'

17.2 Compaction Boundary Messages

```
// Create compaction boundary
createCompactBoundaryMessage(
  trigger: 'auto' | 'manual',
  preTokens: number,
  lastPreCompactMessageUuid?,
  userContext?,
  messagesSummarized?,
): SystemCompactBoundaryMessage

// Result structure:
{
  type: 'system',
  subtype: 'compact_boundary',
  content: 'Conversation compacted',
  compactMetadata: {
    trigger: 'auto' | 'manual',
    preTokens: number,
    userContext?: string,
    messagesSummarized?: number,
    preservedSegment?: {
      headUuid: UUID,
      anchorUuid: UUID,
      tailUuid: UUID,
    },
    preCompactDiscoveredTools?: string[],
  },
```

```
  logicalParentUuid?: UUID,  // Points to the last message before compaction
}
```

Key fields:

- `logicalParentUuid` : Establishes the relationship between the compaction boundary and the last message before compaction, used for session storage's chain recovery
- `preservedSegment` : Tells the loader how to restore the preserved segment when deduplication/skipping
- `preCompactDiscoveredTools` : Deferred tool schemas loaded before compaction

17.3 Microcompact Boundary Messages

```
createMicrocompactBoundaryMessage(
  trigger: 'auto',
  preTokens: number,
  tokensSaved: number,
  compactedToolIds: string[],
  clearedAttachmentUUIDs: string[],
): SystemMicrocompactBoundaryMessage
```

17.4 Finding Compaction Boundaries

```
// Find the index of the last compaction boundary message
findLastCompactBoundaryIndex(messages): number

// Get all messages after the compaction boundary
getMessagesAfterCompactBoundary(messages, options?): T[]
// Includes the boundary message itself
// Supports HISTORY_SNIP: snip view also applies this slice
```

---

18. Key Design Decisions

18.1 Why Fork a Sub-Agent for Summary?

All background tasks (summary generation, memory extraction) are executed via `runForkedAgent` , with core reasons:

1. **Prompt cache reuse**: Fork inherits the main session's complete cache prefix (system prompt + tools + context messages), avoiding redundant computation
2. **State isolation**: Sub-agent cannot modify the main agent's `readFileState` , skill list, etc.
3. **Interruptible**: User Esc interrupts the sub-agent via shared `AbortController` signal
4. **No side effects**: Sub-agent's messages don't enter the main transcript (`skipTranscript: true`)

18.2 Why Microcompact at the API Layer Instead of Directly Modifying Messages?

Cached Microcompact chooses to operate at the API layer via `cache_edits` rather than directly modifying message content:

1. **Preserve prompt cache**: Directly modifying message content breaks the server-side cache, causing recalculation every time

2. **Reduce network load**: API only transmits incremental changes (what to delete), not the entire prefix
3. **Atomicity**: Server-side cache operations are atomic, avoiding race conditions
4. **Complementary to compaction**: Microcompact cleans old results, full compaction handles accumulated pressure

18.3 Why Use GrowthBook Dynamic Configuration?

All thresholds and switches are dynamically configured via GrowthBook rather than hardcoded:

```
getFeatureValue_CACHED_MAY_BE_STALE('tengu_session_memory', false)
getDynamicConfig_CACHED_MAY_BE_STALE('tengu_sm_compact_config', {})
```

- **Zero-delay deployment**: Adjust thresholds without releasing new versions
- **A/B testing**: Different user groups use different configurations
- **Gradual rollout**: Expand feature coverage progressively
- **Non-blocking cache**: `CACHED_MAY_BE_STALE` immediately returns cached value, doesn't block initialization

PS: These parameter thresholds and switches are directly exposed interfaces that can be adjusted without modifying the underlying code.

---

19. Global Data Flow Overview

19.1 Complete Request Processing Flow

Where is session memory compact??

```
User Input
    │
    ▼
┌──────────────────────────────────────────────────────────────────────┐
│ 1. Microcompact                                                      │
│    microcompactMessages(messages, toolUseContext, querySource)        │
│                                                                      │
│    ┌──────────────────┐    ┌──────────────────────────────────┐    │
│    │ Time-based MC    │    │ Cache Edit MC (CACHED_MICROCOMPACT) │   │
│    │ if (gap >= 60min) │    │ if (cache_edits API available)    │   │
│    │ Clean old tool results │    │ Delete old tools, keep cache prefix    │   │
│    └──────────────────┘    └──────────────────────────────────┘    │
│                                                                      │
│    └─ Return { messages (cleaned), compactionInfo: { pendingCacheEdits } } │
└──────────────────────────────────────────────────────────────────────┘
    │
    ▼
┌──────────────────────────────────────────────────────────────────────┐
│ 2. Snip History Pruning (HISTORY_SNIP feature) -> This is actually disabled │
│    snipCompactIfNeeded(messages)                                      │
│    ├─ Calculate snip point (by token count or message count)         │
│    ├─ Replace with SNIP_MARKER                                       │
│    └─ Return tokensFreed (passed to autocompact)                     │
└──────────────────────────────────────────────────────────────────────┘
    │
```

```
    ▼
┌──────────────────────────────────────────────────────────────────────┐
│ 3. Token Counting and Threshold Detection                            │
│    tokenCountWithEstimation(messages) - snipTokensFreed               │
│    vs getAutoCompactThreshold(model)                                  │
│                                                                      │
│    ┌──────────────┐  ┌──────────────────┐  ┌────────────────────────┐  │
│    │ < threshold  │  │ >= threshold & < │  │ >= blocking limit   │  │
│    │ Normal API call │  │ blocking        │  │ Block input         │  │
│    │              │  │ Trigger warning/error │ │                    │  │
│    └──────────────┘  └──────────────────┘  └────────────────────────┘  │
└──────────────────────────────────────────────────────────────────────┘
    │
    ▼ (exceeds threshold)
┌──────────────────────────────────────────────────────────────────────┐
│ 4. Session Memory Compact / Auto-Compact                             │
│    autoCompactIfNeeded(messages, toolUseContext, ...)                 │
│                                                                      │
│    ┌────────────────┐    ┌──────────────────────────────────────┐    │
│    │ SM-Compact?    │    │ Traditional compaction            │    │
│    │ (priority use) │    │ compactConversation()             │    │
│    │ No API call    │    │ ├─ fork sub-agent to generate summary │  │
│    └────────────────┘    │ ├─ clear file read cache             │    │
│                          │ ├─ reinject files/skills/plan       │    │
│                          │ ├─ create compaction boundary       │    │
│                          │ └─ PostCompact hooks + cleanup      │    │
│                          └──────────────────────────────────────┘    │
│                                                                      │
│    Success → reset circuit breaker consecutiveFailures = 0           │
│    Failure → consecutiveFailures++ → circuit break after 3 times     │
└──────────────────────────────────────────────────────────────────────┘
    │
    ▼
┌──────────────────────────────────────────────────────────────────────┐
│ 5. API Call (queryModelWithStreaming)                                │
│    ├─ normalizeMessagesForAPI()                                      │
│    ├─ inject cache_reference + cache_edits (if any)                   │
│    ├─ send anthropic.beta.messages.create streaming request           │
│    └─ process streaming response                                     │
└──────────────────────────────────────────────────────────────────────┘
    │
    ▼
┌──────────────────────────────────────────────────────────────────────┐
│ 6. Stop Hooks (background tasks, parallel execution)                  │
│                                                                      │
│    ┌──────────────────┐  ┌──────────────────────────────────────┐  │
│    │ extractMemories  │  │ extractSessionMemory (sessionMemory) │  │
│    │ → Persistent memory│ │ → Update session memory Markdown    │  │
│    │ → memory/ directory │ │ → ~/.claude/sessions/<id>/         │  │
│    └──────────────────┘  └──────────────────────────────────────┘  │
│                                                                      │
│    Mutual exclusion: if main agent already wrote memory, skip extraction │
└──────────────────────────────────────────────────────────────────────┘
```

```
    │
    ▼
  User sees response
```

19.2 Compaction Message Construction Flow

```
CompactionResult
    │
    ├─ boundaryMarker        ← SystemMessage(type=compact_boundary)
    │       └─ compactMetadata: { trigger, preTokens, preservedSegment }
    │
    ├─ summaryMessages[]      ← UserMessage(isCompactSummary=true)
    │       └─ isVisibleInTranscriptOnly: true (not in API)
    │
    ├─ messagesToKeep[]       ← Most recent N messages (SM-Compact scenario)
    │
    ├─ attachments[]         ← AttachmentMessage[]
    │       ├─ postCompactFileAttachments (recently read files, max 5, total 50K)
    │       ├─ taskStatus (async agent state)
    │       ├─ plan_file_reference (Plan file)
    │       ├─ plan_mode (Plan Mode reminder)
    │       ├─ invoked_skills (used Skills, max 5K each)
    │       └─ deferredToolsDelta / agentListingDelta / mcpInstructionsDelta
    │
    └─ hookResults[]          ← HookResultMessage[]
            └─ results from processSessionStartHooks()
                └─ reload CLAUDE.md, memory files, etc.

Assembly: buildPostCompactMessages(result)
    [boundaryMarker, ...summaryMessages, ...messagesToKeep, ...attachments, ...hookResults]
```

19.3 Token Consumption by Context Layer

```
┌──────────────────────────────────────────────────────────────────────┐
│                          Model Context Window                        │
│                                                                      │
│  ┌──────────────┐ ┌──────────────┐ ┌──────────┐ ┌───────────────┐ │
│  │ System prompt│ │ Tool definitions│ │ User context │ │ Message history│ │
│  │ ~30-50K     │ │ ~5-20K       │ │ ~5K      │ │ ~variable    │ │
│  └──────────────┘ └──────────────┘ └──────────┘ └───────────────┘ │
│                                                                      │
│  Message history composition:                                        │
│  ├─ Compaction boundary messages (fixed ~200B)                      │
│  ├─ Summary messages (structured text ~1-5K)                        │
│  ├─ Preserved tail messages (most recent N, 10K-40K)                │
│  ├─ File attachments (max 5 files, 5K/file)                         │
│  ├─ Skills attachments (max 25K total)                              │
│  ├─ Tool results after microcompact cleanup (only most recent)      │
│  └─ Latest user message + assistant response                        │
└──────────────────────────────────────────────────────────────────────┘
```

Appendix A: Key File Index

File Path	Description
`utils/tokens.ts`	Token counting (precise + estimation), API usage extraction
`utils/context.ts`	Context window constants and model-related calculations
`utils/contextAnalysis.ts`	Context statistics analysis, duplicate read detection
`services/tokenEstimation.ts`	Rough token estimation (with special type handling)
`services/compact/microCompact.ts`	Microcompact entry, time-triggered + cache editing
`services/compact/timeBasedMCConfig.ts`	Time-triggered MC GrowthBook configuration
`services/compact/apiMicrocompact.ts`	API layer context management strategies (clear_tool_uses, clear_thinking)
`services/compact/autoCompact.ts`	Auto-compact trigger, circuit breaker, threshold system
`services/compact/compact.ts`	Full compaction core logic (summary generation, file recovery)
`services/compact/prompt.ts`	Compaction summary prompt template
`services/compact/grouping.ts`	API round message grouping (by message.id)
`services/compact/postCompactCleanup.ts`	Post-compaction cache cleanup
`services/compact/compactWarningState.ts`	Compaction warning state management
`services/compact/compactWarningHook.ts`	React hook: subscribe to warning suppression state
`services/compact/sessionMemoryCompact.ts`	Session memory compaction (SM-Compact)
`services/SessionMemory/sessionMemory.ts`	Session memory extraction and update
`services/SessionMemory/sessionMemoryUtils.ts`	Session memory utilities, threshold calculation
`services/SessionMemory/prompts.ts`	Session memory template and prompt generation
`services/extractMemories/extractMemories.ts`	Persistent memory extraction
`context.ts`	System context (Git status), user context (CLAUDE.md)
`history.ts`	REPL prompt history management
`utils/messages.ts`	Message normalization, boundary messages, utilities
`utils/sessionStorage.ts`	Session message persistence
`services/contextCollapse/index.js`	Context Collapse (experimental, dynamically loaded)

Appendix B: All Configurable Constants

Constant	Value	File
`MODEL_CONTEXT_WINDOW_DEFAULT`	200,000	`utils/context.ts`
`COMPACT_MAX_OUTPUT_TOKENS`	20,000	`utils/context.ts`
`AUTOCOMPACT_BUFFER_TOKENS`	13,000	`services/compact/autoCompact.ts`
`WARNING_THRESHOLD_BUFFER_TOKENS`	20,000	`services/compact/autoCompact.ts`
`MANUAL_COMPACT_BUFFER_TOKENS`	3,000	`services/compact/autoCompact.ts`
`MAX_CONSECUTIVE_AUTOCOMPACT_FAILURES`	3	`services/compact/autoCompact.ts`
`POST_COMPACT_MAX_FILES_TO_RESTORE`	5	`services/compact/compact.ts`
`POST_COMPACT_TOKEN_BUDGET`	50,000	`services/compact/compact.ts`

Constant	Value	File
`POST_COMPACT_MAX_TOKENS_PER_FILE`	5,000	`services/compact/compact.ts`
`POST_COMPACT_MAX_TOKENS_PER_SKILL`	5,000	`services/compact/compact.ts`
`POST_COMPACT_SKILLS_TOKEN_BUDGET`	25,000	`services/compact/compact.ts`
`MAX_PTL_RETRIES`	3	`services/compact/compact.ts`
`MAX_SECTION_LENGTH`	2,000	`services/SessionMemory/prompts.ts`
`MAX_TOTAL_SESSION_MEMORY_TOKENS`	12,000	`services/SessionMemory/prompts.ts`
`DEFAULT_SM_COMPACT_CONFIG.minTokens`	10,000	`services/compact/sessionMemoryCompact.ts`
`DEFAULT_SM_COMPACT_CONFIG.maxTokens`	40,000	`services/compact/sessionMemoryCompact.ts`
`MINIMUM_MESSAGETOKENS_TO_INIT`	10,000	`services/SessionMemory/sessionMemoryUtils.ts`
`MINIMUM_TOKENS_BETWEEN_UPDATE`	5,000	`services/SessionMemory/sessionMemoryUtils.ts`
`TOOL_CALLS_BETWEEN_UPDATES`	3	`services/SessionMemory/sessionMemoryUtils.ts`
`TIME_BASED_MC_CONFIG.gapThresholdMinutes`	60	`services/compact/timeBasedMCConfig.ts`
`DEFAULT_MAX_INPUT_TOKENS`	180,000	`services/compact/apiMicrocompact.ts`
`DEFAULT_TARGET_INPUT_TOKENS`	40,000	`services/compact/apiMicrocompact.ts`
`IMAGE_MAX_TOKEN_SIZE`	2,000	`services/compact/microCompact.ts`
`MAX_HISTORY_ITEMS`	100	`history.ts`
`MAX_STATUS_CHARS`	2,000	`context.ts`
`TOKEN_COUNT_THINKING_BUDGET`	1,024	`services/tokenEstimation.ts`